

Contents

AeroMorse — Complete Build Guide

USB HID Keyboard & Mouse via Sip-and-Puff or Accessibility Switches

Table of Contents

1. What AeroMorse Does

Morse code basics

2. How This Guide Works

3. Compatible Feather Boards

Why not every Feather works

BLE HID note — CircuitPython 10.x

5477). Always confirm on your board's circuitpython.org page under

Boards that work

Metro form factor boards

Boards that do NOT work for USB HID

Recommended board: Adafruit ESP32-S3 Reverse TFT Feather #5691

Building with no header pins — what works

4. Input Method Options

Option A — Sip-and-Puff (LPS33HW pressure sensor)

Option B — Accessibility (AT) Switches via TRRS Jack

Option B1 — Solderless breadboard

Option B2 — Soldered TRRS breakout (compact, durable)

Option B3 — #2915 TRRS Terminal Block (no breadboard, minimal soldering) ★ Recommended for #5691

5. Display Options

Understanding the columns

Built-in display — no separate display needed

FeatherWing displays (plug directly on Feather — easiest, requires headers)

Standalone / breakout displays (5 wires required)

EYESPI displays (advanced — SPI wiring via breakout adapter)

OLED displays (STEMMA QT plug-and-play, monochrome, no colour)

Wireless Display — ESP-NOW Remote Mirror

Option W1 — Second ESP32-S3 Reverse TFT Feather #5691 ★ Recommended

Option W2 — Adafruit MagTag (2.9" e-ink, plug-and-play)

Wireless display comparison

Display driver library required per display chip

6. Speaker Options

Option S1 — STEMMA Speaker (easiest) ★ Recommended

Option S2 — Passive piezo with detachable 3.5 mm connection (solderless)

Parts

Wiring

Connection table

Why this works without signal conditioning

Already have a 3.5 mm-plug piezo or speaker?

2790 plug — the existing plug goes straight into the #2915 jack. You

Common configurations — how many #2915 jacks you need

Wiring guide for each role

2790 plug-side terminal block — that gives you a fully solderless

Connecting either jack to the Feather

Option S3 — Small speaker + separate amplifier board

7. Complete Parts Lists

Core hardware (always required)

Input method — choose ONE

Sip-and-Puff sensor parts

AT switch — solderless breadboard

AT switch — soldered TRRS breakout

AT switch — #2915 TRRS Terminal Block (no breadboard) ★ Recommended for #5691

Display — choose ONE

Wireless display — choose ONE (optional, for remote viewing)

Option W1 — Second Reverse TFT Feather (recommended)

Option W2 — Adafruit MagTag (e-ink, plug-and-play)

Speaker — choose ONE (optional but recommended)

Tools

Hardware tools — solderless build (Option B1 breadboard)

Hardware tools — soldered build (Option B2 TRRS breakout, or soldering headers)

Useful for any build (optional but handy)

8. Step-by-Step Assembly

Before you start

Step 8A — Connect the display

If you chose a FeatherWing display (#3651, #5872, or #3315)

If you chose the OLED #326

If you chose a standalone TFT breakout (#2050, #1770, etc.)

Step 8B — Connect the pressure sensor (sip-and-puff only)

Step 8C — Connect the AT switches (switch mode only)

Option B1 — Breadboard

Option B2 — TRRS breakout (soldered)

Option B3 — #2915 TRRS Terminal Block

Step 8D — Connect the speaker

If your Feather has header pins (or sits in a breadboard)

If your Feather has no header pins (bare PCB pads)

Quick reference — all paths, all options

Step 8E — Set up the wireless display board hardware (optional)

Option W1 — Second Reverse TFT Feather

Option W2 — Adafruit MagTag (#4800)

9. Software Installation

Step 9.1 — Install CircuitPython on the Feather

Step 9.2 — Install the required libraries

Step 9.3 — Install Thonny and watch the serial console

Step 9.3.1 — Download and install Thonny

Step 9.3.2 — Confirm the Feather is plugged in

Step 9.3.3 — Point Thonny at CircuitPython

Step 9.3.4 — Leave the Shell panel visible

Step 9.3.5 — Capturing a fresh boot trace (Ctrl+D)

Useful Thonny shortcuts

Step 9.4 — Copy the AeroMorse project files

Step 9.4.0 — Download the AeroMorse software (do this first)

👉 <https://github.com/jlubin2001/AeroMorse>

Step 9.4.1 — Software for the Wireless Display receiver (optional)

Files on the receiver

Firmware on the receiver (Step 9.1 equivalent)

Option W1 — Second ESP32-S3 Reverse TFT Feather #5691 (recommended)

Option W2 — Adafruit MagTag (#4800, 2.9" e-ink)

Verifying the receiver

10. Configuration

Key settings — quick reference

Input modes — what SWITCH_MODE does

11. First Power-On Test

Quick test — sensor mode

Quick test — switch mode

If nothing appears

12. Troubleshooting

Input — sip-and-puff sensor (#4414)

Input — AT switches

Input — timing

Display

Speaker

Wireless display (ESP-NOW)

Software / general

Appendix A — Groups and Group Cycling

Appendix B — Display Layout

Appendix C — Customising Macros

Appendix D — Breadboard Wiring Walkthrough

Parts — exactly what to buy

Understanding the breadboard

Step 1 — Place the Feather in the breadboard

Step 2 — Place the 3.5 mm stereo jack (#1699)

Step 3 — Connect with jumper wires

Step 4 — Connect your AT switches

Appendix E — Configuration Reference

Input — sensor / switches / thresholds

4414) — sip is a dot, puff is a dash. False selects Option B (two

Input mode — 1 / 2 / 3 switch

Strong sip / strong puff

Timing

Code repeat (Darci-style hold-to-repeat)

Audio

Mouse (Group 2 movement speeds)

Repeat exclusions

Display / wireless

Appendix F — Building with #5477 + 3.5" TFT FeatherWing #3651

F.1 When this build is the right pick

F.2 Parts list

F.3 Header pins on the #5477 — required

5477 ships as a bare PCB with empty through-hole pads. The

F.4 Hardware assembly

F.5 Software setup

Libraries on /lib

Project files

Edit 1 — Display init swap (required)

Edit 2 — Resize labels and reposition the pressure bar (optional but strongly recommended)

F.6 Configuration

F.7 Verifying it works

AeroMorse — Complete Build Guide

USB HID Keyboard & Mouse via Sip-and-Puff or Accessibility Switches

Software & this guide live here: <https://github.com/jlubin2001/AeroMorse> — always download the firmware (`code.py` , `boot.py` , `morse_map.py` , `config.py`) from that repo. See **§9.4** for the download steps.

This guide walks a first-time, non-technical builder through every decision and every step needed to assemble a working AeroMorse device. Read the whole guide once before buying anything — your hardware choices affect each other.

Table of Contents

1. [What AeroMorse Does](#)
 2. [How This Guide Works](#)
 3. [Compatible Feather Boards](#)
 4. [Input Method Options](#)
 5. [Display Options - Wireless Display — ESP-NOW Remote Mirror](#)
 6. [Speaker Options](#)
 7. [Complete Parts Lists](#)
 8. [Step-by-Step Assembly](#)
 9. [Software Installation](#)
 10. [Configuration](#)
 11. [First Power-On Test](#)
 12. [Troubleshooting](#)
-

1. What AeroMorse Does

AeroMorse is an open-source CircuitPython project directed by Jim Lubin — a ventilator-dependent quadriplegic who has used Morse code for computer access since 1989. Inspired by [AirTalker](#), it turns an Adafruit Feather microcontroller into a USB HID keyboard and mouse that connects via USB-C and appears to the host as a standard keyboard and mouse with no drivers required. Works on **Windows, macOS, Linux, iPadOS, Android, and ChromeOS**.

How this project was built — what's confirmed vs documented. Jim is the user, project lead, and source of all design decisions: hardware choices, input-mode requirements, Morse code-set conventions,

accessibility trade-offs, and ongoing user feedback (his own and from other AAC users — Darci USB veterans in particular). The firmware (`code.py`), this build guide, and the comparison documents were written by **Claude Opus 4.7** (Anthropic) acting as the coding assistant — a "vibe coding" workflow in which Jim directs and Claude writes. Jim does not write the firmware himself, and has not personally soldered or assembled every hardware combination listed here. Several options — particularly some board / display / speaker combinations — are documented from datasheets and Claude's understanding of the parts rather than from a verified build.

If you build a configuration, please report back via a GitHub issue — whether it works or doesn't.

Confirmed-vs-theoretical is the single most useful signal this project can collect right now.

Input is by **sip-and-puff** (LPS33HW pressure sensor) or **two standard AT switches**. A short sip (or switch 1) is a dot; a short puff (or switch 2) is a dash. A small OLED display shows the active group, the Morse pattern as it builds, and the last action. An optional speaker beeps for every dot and dash.

Ten groups organize all functions — `g0` plus `g1-g9` :

- **Group 0** — always-available system layer; 8-symbol patterns jump directly to any other group from anywhere
- **Group 1 — Keyboard**: letters, numbers, punctuation, function keys, navigation, sticky modifiers (default group at boot)
- **Group 2 — Mouse**: movement, clicks, drag, repeat, and Windows shortcuts
- **Group 3 — Macro**: user-defined text strings
- **Group 4 — Scanning**: Enter, Space and F3–F12 on the 12 shortest codes — for iOS / Android Switch Control
- **Group 5 — Media**: USB HID Consumer Controls — play/pause, volume, mute, track skip, brightness, plus launchers for calculator, file explorer, browser, and mail
- **Group 7 — Switch**: no Morse — a sip holds Enter and a puff holds Space for as long as you keep going, like two real switches (games and switch apps). See [AEROMORSE_SWITCH_MODE_GUIDE.md](#)
- **Groups 6, 8, 9 — Placeholders** seeded with g1's letters and numbers, ready for you to customise

Groups cycle with a long sip or puff. An optional **ESP-NOW wireless display** mirrors the main screen on a second board up to ~30 m away — useful when the sensor is mounted behind the user.

Existing **Darci USB** users can drop in `morse_map_darci.py` to use their familiar code set. All Morse assignments are fully customizable in `morse_map.py` . Parts cost approximately **\$50–\$100** in off-the-shelf components.

Morse code basics

Symbol	Input	Display
Dot	Short sip or press	.
Dash	Short puff or press	-
Letter gap	Pause ≥ 0.2 s	(pattern fires)

Example: sip · puff · puff = . - - = the letter **W**

You do not need to know Morse code from memory to begin — the device can be used with a printed reference card while you learn.

2. How This Guide Works

AeroMorse has modular hardware. You pick:

1. **A Feather microcontroller board** (the brain — required)
2. **An input method** (sip-and-puff sensor or AT switches — required)
3. **A display** (what you see — optional but recommended)
4. **A speaker** (audio feedback — optional)

Once you have chosen each, jump to the relevant wiring section in Step 8.

Recommended combination for most builders (USB HID, display included): ESP32-S3 Reverse TFT Feather #5691 · LPS33HW sensor #4414 · STEMMA Speaker #3885

If you need Bluetooth wireless HID: Metro ESP32-S3 #5500 · 1.3" OLED #938 · LPS33HW sensor #4414 · STEMMA Speaker #3885

3. Compatible Feather Boards

Why not every Feather works

AeroMorse uses **USB HID** — the USB standard that lets keyboards and mice talk to computers. Only Feather boards with **native USB** support this. Classic ESP32 boards (including Adafruit #5900 Feather ESP32 V2) use a separate USB-to-serial chip that cannot emulate a keyboard. Those boards will not work.

BLE HID note — CircuitPython 10.x

The chip supports it, but CircuitPython's BLE keyboard support on the ESP32-S3 is still not reliable — tested, see below. The ESP32-S3 hardware fully supports Bluetooth LE; the problems are in CircuitPython's ESP32 BLE code. Several were fixed (issues #9430, #9669, and #10739, the last marked fixed in 10.3.0), but on CircuitPython 10.3.1 a BLE keyboard still does not stay connected to an Android phone.

Tested 2026-09-26 — #5691 with TinyUF2 0.35.0 + CircuitPython 10.3.1, host a Samsung Galaxy A15 (Android 16), using `test_ble.py` from this repo: - The board advertises and the phone pairs (`paired=True`). - **No keystrokes ever reached the phone** — not on any connection, even while the board reported it was paired — although CircuitPython raised no error when sending them. - The link then **drops about 6.5 s after every connection**, and every reconnect starts **unpaired** again — the board doesn't keep the pairing — so it loops connect → pair → drop.

Result: not usable yet. Reported upstream on [adafruit/circuitpython #10739](https://github.com/adafruit/circuitpython/issues/10739). An iPad or Windows host has not been tested. Run `test_ble.py` to check a newer CircuitPython or a different host. For a phone or tablet today, use a **USB-C cable** — AeroMorse works as a wired keyboard/mouse.

4 MB flash boards include BLE on 10.x. Earlier builds sometimes left the BLE stack out of 4 MB firmware images to save space. As of **CircuitPython 10.3.1**, circuitpython.org lists the `_bleio` (BLE) module — together with `espnw` and `usb_hid` — for all three 4 MB boards below (#5691, #5483,

5477). Always confirm on your board's circuitpython.org page under

"Built-in modules available".

Before you can use BLE: - **You need CircuitPython 10.x** — BLE HID is not reliable on 9.x. - **An older bootloader may refuse to flash 10.x.** A #5691 with the 2023 TinyUF2 bootloader (0.12.3) accepted 9.2.9 but silently ignored a 10.x `.uf2` — the drive stayed `FTHS3B00T` and never rebooted. Update the TinyUF2 bootloader first, then flash 10.x. - **Use the matching 10.x library bundle** in `lib/`.

AeroMorse status: `code.py` sends **USB HID only**. BLE HID (pairing with a phone or iPad) is not implemented, and won't be until the CircuitPython problem above is resolved. Running the ESP-NOW wireless display and BLE at the same time shares one 2.4 GHz radio and has not been tested either. iPads and phones with USB-C already accept AeroMorse as a **wired** USB keyboard/mouse.

Boards that work

Board	Adafruit #	USB HID	BLE HID	PSRAM	WiFi	Notes
ESP32-S3 Reverse TFT Feather	#5691	✓	✓✘	✓ 2MB	✓	Built-in 1.14" TFT — screen faces down (panel mount). Recommended for panel-mount enclosures — see below. Not great on a breadboard once headers are soldered (TFT ends up pressed against the breadboard).
ESP32-S3 TFT Feather	#5483	✓	✓✘	✓ 2MB	✓	Built-in 1.14" TFT — screen faces up. Best built-in-TFT option if you plan to use a breadboard with headers.
ESP32-S3 Feather 4MB/2MB PSRAM	#5477	✓	✓✘	✓ 2MB	✓	No built-in display — pair with a STEMMA QT OLED (#326 / #938) or a FeatherWing TFT (\$5). EYESPI is also possible but needs the most code editing.
ESP32-S2 TFT Feather	#5300	✓	—	✓ 2MB	✓	Built-in 1.14" TFT — older S2 chip; no BLE

✘ BLE is included in CircuitPython 10.3.1 for these boards (needs 10.x and a bootloader that can flash it), but a BLE keyboard does **not** stay connected reliably yet — see the tested result in the BLE HID note above. AeroMorse firmware does not use BLE. The ESP32-S2 (#5300) has no Bluetooth hardware.

Metro form factor boards

Metro boards use the Arduino Uno footprint (68×53 mm) — larger than a Feather. They work for AeroMorse but with one key difference: **FeatherWing displays will not plug directly onto them.** Use a standalone TFT breakout (SPI wires) or the STEMMA QT OLED instead. The pressure sensor plugs straight in via STEMMA QT with no wiring change.

Board	Adafruit #	USB HID	BLE HID	PSRAM	WiFi	Notes
Metro ESP32-S3	#5500	✓	✓	✓ 8MB	✓	Most memory (16 MB flash + 8 MB PSRAM); built-in LiPoly charging + battery monitor

The Metro ESP32-S3 #5500 has the most memory in this guide (16 MB flash, 8 MB PSRAM). BLE is **not** unique to it — on CircuitPython 10.x the 4 MB Feathers (#5691 / #5483 / #5477) include BLE too. Choose the Metro for the extra memory or the larger form factor.

Boards that do NOT work for USB HID

Board	Reason
Feather ESP32 V2 #5400	No native USB — uses CP2102N serial chip
Feather ESP32 Huzzah #3405	No native USB
Feather ESP8266 #2821	No native USB

Recommended board: Adafruit ESP32-S3 Reverse TFT Feather #5691

Why this one:

- **Display already built in** — the 240×135 colour TFT is soldered to the board, facing down for panel mounting; no separate display to buy or wire
- Has **PSRAM** (2 MB extra RAM) — handles all display and sensor work without running out of memory
- Has a **STEMMA QT** port — the LPS33HW pressure sensor plugs straight in with no soldering
- **WiFi + ESP-NOW** built in — supports the wireless remote display option (Section 5) without any extra hardware on the main board
- USB HID works out of the box — appears as keyboard and mouse to any computer
- CircuitPython support is excellent and actively maintained
- Available from Adafruit: <https://www.adafruit.com/product/5691>

Watch out — #5691 is not breadboard-friendly once you solder headers. The TFT is on the back of the board, so once header pins are soldered and the Feather is pressed into a breadboard, the screen ends up facing the breadboard and is unreadable. #5691 is designed for **panel-mount enclosures** (a cutout in a case with the screen showing through), not for prototyping on a solderless breadboard. If you plan to develop on a breadboard before moving to an enclosure, use **#5483** (built-in TFT facing up — screen still visible when the Feather is on a breadboard) or **#5477** with an external display.

If you want a built-in TFT and plan to use a breadboard (or no enclosure):

The **ESP32-S3 TFT Feather #5483** is electrically identical to #5691, but its TFT faces the same side as the components. Press it into a breadboard with headers and the screen still faces up. Everything else — PSRAM, STEMMA QT, ESP-NOW, USB HID — is the same. <https://www.adafruit.com/product/5483>

If you want a bare Feather (no built-in display) for the cleanest breadboard or for maximum display flexibility:

The **ESP32-S3 Feather #5477** is the same chip without a soldered-on TFT, so it sits flat in a breadboard. Pair it with one of:

- **STEMMA QT OLED #326 (0.96") or #938 (1.3")** — plug-and-play via the STEMMA QT chain (the same chain that carries the sensor). You'll need to swap the `display = board.DISPLAY` line at the top of `code.py` for a short SSD1306 init block (a few lines). Monochrome 128×64.
- **FeatherWing TFT (#3651, #5872, or #3315)** — plugs straight onto the header pins, no extra wiring. Same kind of one-time `code.py` display-init swap. Larger, colour, much more screen area than the OLED. For the full **#5477 + #3651** combo see **Appendix F** at the end of this guide for a consolidated walkthrough including the display-init code to paste, the upscaled-label layout for 480×320, and the pressure-bar repositioning.
- **EYESPI TFT** (Build Guide §5 "EYESPI displays") — possible but the most involved option: needs a #5613 breakout, ~7 jumper wires, a flex cable, and a more complex `displayio` init block. Use only if you specifically want EYESPI's flex-cable advantages.

<https://www.adafruit.com/product/5477>

If you want the most memory (Metro form factor):

The **Metro ESP32-S3 #5500** has the most memory in this guide (8 MB PSRAM, 16 MB flash). Like the Feathers above, it supports USB HID and includes BLE on CircuitPython 10.x. The extra memory does not fix the BLE keyboard problem — that's in CircuitPython's ESP32 BLE code (see the tested result in the BLE HID note in §3), and AeroMorse firmware does not use BLE.

The #5500 has no built-in display — connect the STEMMA QT OLED or a standalone TFT breakout. The ESP-NOW wireless display (Section 5) works on it identically to the Feather boards.

<https://www.adafruit.com/product/5500>

A note on PSRAM:

PSRAM is extra RAM soldered alongside the main chip. The ESP32-S3 chip itself has 512 KB of built-in RAM; a colour TFT display needs a *framebuffer* (a block of RAM holding every pixel) that can easily exceed that. Every Feather and Metro board still listed above has at least **2 MB of PSRAM** (the #5500 Metro has 8 MB), so the framebuffer lives in PSRAM and the built-in 512 KB stays free for code. Any supported display size — 128×64 OLED through 480×320 FeatherWing — fits comfortably.

Quick decision guide:

I want...	Choose
Simplest build, colour TFT included, USB only	#5691 Reverse TFT Feather
Same but screen faces up	#5483 TFT Feather
BLE HID (in addition to USB HID)	Not reliable yet on ESP32-S3 / CircuitPython (see the BLE HID note in §3) — use a USB-C cable to phones/tablets for now
Most memory (8 MB PSRAM)	#5500 Metro ESP32-S3

Building with no header pins — what works

Many Feather boards ship as bare PCBs with no header pins soldered. If you don't want to solder headers (and your supplier didn't pre-install them), some build paths still work via STEMMA QT plug-and-play, some require soldering a few wires directly to the Feather's PCB pads, and some fundamentally need headers.

Component / option	Works without headers?	What's needed
#5691 built-in TFT display	✓ Plug-and-play	Display soldered to board — nothing to wire
STEMMA QT OLED (#326, #938)	✓ Plug-and-play	STEMMA QT cable, no Feather wiring
STEMMA QT sensor (#4414 LPS33HW)	✓ Plug-and-play	STEMMA QT cable
FeatherWing displays (#3651, #5872, #3315)	X Not possible	The wing's holes can't grip bare pads — headers required
Standalone TFT breakouts (#2050, #1770...)	⚠ Solder 5–7 wires	Solder display wires directly to Feather pads
AT switches — Option B1 breadboard	X Not possible	Feather must sit <i>in</i> the breadboard via headers
AT switches — Option B2 TRRS breakout	⚠ Solder 3 wires	Solder breakout output wires to D5/D6/GND pads
AT switches — Option B3 #2915 Terminal Block	⚠ Solder 3 wires	Solder terminal-block wires to D5/D6/GND pads
Speaker — #3885 STEMMA Speaker	⚠ Solder 3 wires	Cut JST PH plug off, solder to A0/3V/GND pads
Speaker — Option S2 piezo (#2915 + #2790 + #1740/#1739)	⚠ Solder 2 wires only if no headers	If Feather has headers: 2 F-F jumpers from #2915 → A0/GND (zero soldering). If no headers: solder 2 wires from #2915 → A0/GND pads. Piezo side is screw-terminal either way.
Speaker — PAM8302 amp (#2130)	⚠ Solder 3 wires	Solder amp board to A0/3V/GND pads

STEMMA QT does not solve every problem. STEMMA QT carries I²C, 3 V, and GND only — that is enough for sensors and I²C displays (OLEDs), but **not for audio** (analog signal must come from A0) or **arbitrary switches** (digital inputs need GPIO pins).

An I²C GPIO expander board (AW9523 #4886, MCP23017 #5346, PCF8575 #5611) *could* host AT switches via STEMMA QT, but you still need to wire jacks to the expander's GPIO pads — and `code.py` would need a new library and rewritten input routines. **More work, no benefit** vs. soldering three wires directly to D5/D6/GND on the Feather.

Recommended fully-header-less build:

- Feather: **#5691** (built-in TFT — no wiring)
- Sensor: **#4414 LPS33HW** via STEMMA QT (or skip if using AT switches)
- AT switches: **#2915 Terminal Block** + 3 wires soldered to D5/D6/GND pads

- Speaker: 3 wires soldered to A0/3V/GND pads (or skip — display alone is fine)

Total soldering for the full build: **3–6 wires to Feather pads** (skip the sensor row if you use switches, skip both if you don't want a speaker). Each pad is ~1.5 mm — much easier to solder than the pin holes for headers.

No soldering at all? Skip the speaker and use the **#5691 + sensor only** combination. The OLED/TFT shows every dot and dash visually. Cost: a #5691 + #4414 + STEMMA QT cable, ~\$30, zero solder joints. Add a third-party 3.5 mm Y-splitter and your existing AT switches won't work in this config — sensor mode is the no-solder path.

4. Input Method Options

Option A — Sip-and-Puff (LPS33HW pressure sensor)

A small barometric pressure sensor detects the tiny pressure change when you breathe into or out of a short plastic tube. No hand or head movement required.

Sensor: Adafruit LPS33HW Water-Resistant Pressure Sensor — STEMMA QT

<https://www.adafruit.com/product/4414>

Adafruit learn guide — LPS33 sip-and-puff with CircuitPython: <https://learn.adafruit.com/st-lps33-and-circuitpython-sip-and-puff> Useful **background reading** on the sensor — calibration, threshold tuning, and breath technique.

⚠ **Read it for concepts only — do NOT install its code.** That Adafruit project is built around a *different board* and ships its own example software. AeroMorse does **not** use any of it. Your firmware is solely the AeroMorse [code.py](#) / [boot.py](#) / [morse_map.py](#) / [config.py](#) from <https://github.com/jlubin2001/AeroMorse> (see §9.4). Ignore any "install this code" step in the Adafruit guide.

Tube options:

Option	ID	OD	Material	Source
Adafruit Silicone Tubing #3659	2.5 mm	4.7 mm	Food-safe silicone	https://www.adafruit.com/product/3659
Aquarium airline tubing	4.8 mm (3/16")	7.9 mm (5/16")	PVC or silicone	Pet store / hardware store
Medical sip-and-puff tubing	4.8 mm (3/16")	7.9 mm (5/16")	Medical PVC	Medical supply

Adafruit #3659 is the easiest option — sold on the same site as the sensor, food-safe silicone, fits snugly over the sensor's pressure port. Order at least 30 cm (12 inches); 1 m gives you plenty to cut to length and retry.

Aquarium airline tubing is the budget alternative, available at any pet store for ~\$3–5 per metre. The larger outer diameter grips the sensor port slightly differently — wrap a thin layer of silicone tape around the port nipple first if the fit feels loose.

Cut to a comfortable length — most users prefer 20–40 cm (8–16 inches).

The tube pushes over the small nipple on the LPS33HW breakout board. No adhesive is needed; friction holds it securely during normal use.

Option B — Accessibility (AT) Switches via TRRS Jack

Standard assistive technology switches with a **3.5 mm mono plug** connect to the Feather through a TRRS jack breakout. Pressing the dot switch = dot; pressing the dash switch = dash.

Any AT switch compatible with Ablenet, Inclusive Technology, Specs Switches, or similar systems will work. One switch (dot only) is enough to navigate slowly; two switches (dot + dash) are strongly recommended for practical use.

Two wiring options for the TRRS jack:

Option B1 — Solderless breadboard

A no-soldering build using a half-size breadboard, a single TRRS jack (#1699), and three jumper wires. Lowest-cost path and reversible, but bulky and best suited to bench testing rather than long-term use.

Detailed step-by-step breadboard instructions are in [Appendix D — Breadboard Wiring Walkthrough](#) at the end of this guide.

For most builders Option B3 (#2915 Terminal Block) is simpler and tidier — try that first.

Option B2 — Soldered TRRS breakout (compact, durable)

Solders three short wires to an Adafruit TRRS Jack Breakout board (T, R1, S — Ring 2 is unused).

Parts needed: - Adafruit TRRS Jack Breakout #5764 <https://www.adafruit.com/product/5764> - 3 short wires ~8 cm (different colours help) - Soldering iron + solder

TRRS pad	Feather pin	Wire colour suggestion
T (Tip)	D5	Red
R1 (Ring 1)	D6	Blue
S (Sleeve)	GND	Black
R2	<i>(no wire — pad left bare)</i>	—

Both switch options work with the same `code.py` — set `USE_SENSOR = False` in the configuration section.

Option B3 — #2915 TRRS Terminal Block (no breadboard, minimal soldering) ★ Recommended for #5691

Adafruit #2915 — \$2.50 <https://www.adafruit.com/product/2915>

A 3.5 mm TRRS jack with **built-in screw terminals** — no PCB, no soldering on the jack side at all. Strip a wire, insert it, tighten the screw with a small screwdriver. This is the best option for the #5691 Reverse TFT Feather because it avoids the breadboard entirely and does not require the #2926 Terminal Block FeatherWing (which is incompatible with the Reverse TFT Feather).

#2915 terminal	Feather pin	Function
Tip	D5	Dot switch
Ring	D6	Dash switch
Sleeve	GND	Ground
Ring 2	—	Leave empty

If your #5691 has header pins soldered — use female-to-female jumper wires. One end screws into the #2915 terminal; the other end plugs directly onto the Feather pin. **Zero soldering.**

If your #5691 has no header pins — solder the three wires directly to the D5, D6, and GND pads on the Feather. The jack side still needs no soldering. Only 3 solder joints total.

Parts needed:

Qty	Item	Adafruit #	URL
1	TRRS Jack Terminal Block	#2915	https://www.adafruit.com/product/2915
3	Female-to-female jumper wires (if Feather has pins)	#266	https://www.adafruit.com/product/266
—	or 3 short wires ~15 cm + soldering iron (if no header pins)	—	—

Note: The #2915 accepts both mono and TRRS plugs. Most AT switches use a 3.5 mm mono plug (tip + sleeve only), which connects the Tip and Sleeve terminals when pressed — exactly what AeroMorse expects.

5. Display Options

All displays below are supported by CircuitPython's `displayio` system and work with AeroMorse. The code must be updated to match the display driver you choose.

Understanding the columns

- **Form factor:** FeatherWing plugs directly onto the Feather (no wiring). Breakout/standalone requires 5 wires.
- **Interface:** SPI displays have no I²C address conflicts with the sensor.
- **Resolution:** Higher = more text/detail. The Morse buffer, group name, last action, and status line all fit comfortably on 320×240 or larger.

Built-in display — no separate display needed

Board	#	Display size	Resolution	Screen orientation	URL
ESP32-S3 TFT Feather	5483	1.14"	240×135 colour	Faces up — same side as components	https://www.adafruit.com/product/5483
ESP32-S3 Reverse TFT Feather	5691	1.14"	240×135 colour	Faces down — opposite side to components	https://www.adafruit.com/product/5691
ESP32-S2 TFT Feather	5300	1.14"	240×135 colour	Faces up — same side as components	https://www.adafruit.com/product/5300

All three have the display soldered onto the board and accessed via `board.DISPLAY` — no wiring, no extra purchase, no library to add.

Which orientation to choose: - **#5483 (standard)** — screen faces the same direction as the USB port and pins. Best for handheld use or when you want to see the display while working with the board. - **#5691 (reverse)** — screen faces the opposite side. Designed for mounting behind a panel with a cutout so only the screen shows through. Useful if you want to build AeroMorse into an enclosure. - **#5300 (ESP32-S2)** — same orientation as #5483 but older chip. Only choose it if #5483 and #5691 are both unavailable.

All three are functionally identical for AeroMorse.

Code file: #5483, #5691, and #5300 all use the **repo root** `code.py` (which drives the built-in TFT via `board.DISPLAY`).

FeatherWing displays (plug directly on Feather — easiest, requires headers)

Display	#	Size	Resolution	Interface	URL
3.5" TFT Resistive Touch	3651	3.5"	480×320 colour	SPI	https://www.adafruit.com/product/3651
3.5" TFT Capacitive Touch	5872	3.5"	480×320 colour	SPI	https://www.adafruit.com/product/5872
2.4" TFT Resistive Touch	3315	2.4"	320×240 colour	SPI	https://www.adafruit.com/product/3315

Header pins required on the Feather. A FeatherWing has two rows of female header sockets on its underside that slide down onto the male pins on top of the Feather. If your Feather shipped as a bare PCB with empty pin holes, those holes must be filled with soldered male headers before a FeatherWing can be used — there is nothing for the wing to grip otherwise. Adafruit's standard Feather header kit (#2886) is the right part, or order a Feather variant that ships with headers pre-installed. If you cannot or would rather not solder, choose a STEMMA QT OLED (#326 / #938) or the built-in TFT on #5691 instead — both are plug-and-play with no headers.

FeatherWings have a touchscreen controller that uses I²C. You will not use touch input with AeroMorse — the display portion works via SPI independently and will not conflict with the pressure sensor.

Recommended for table mounting: #3651 — largest FeatherWing, bright colour, plugs straight on (once headers are in place) with no additional wiring.

Standalone / breakout displays (5 wires required)

Display	#	Size	Resolution	Driver	URL
3.5" TFT with Touch	2050	3.5"	480×320 colour	HX8357D	https://www.adafruit.com/product/2050
3.2" TFT with Touch	1743	3.2"	320×240 colour	ILI9341	https://www.adafruit.com/product/1743
2.8" TFT with Touch	1770	2.8"	320×240 colour	ILI9341	https://www.adafruit.com/product/1770
2.0" IPS TFT	4311	2.0"	320×240 colour	ST7789	https://www.adafruit.com/product/4311

Standalone displays are useful when you want the display mounted separately from the Feather — for example, a display on an arm at eye level with the Feather tucked out of the way.

EYESPI displays (advanced — SPI wiring via breakout adapter)

EYESPI is Adafruit's standardised **18-pin FPC connector** for SPI displays. Several Adafruit TFT displays have an EYESPI socket, which makes cabling cleaner: instead of individual jumper wires for each SPI signal, a single flat flex cable connects the display to an **EYESPI Breakout Board**.

#5613 — Adafruit EYESPI Breakout Board — 18 Pin FPC Connector — \$1.95

<https://www.adafruit.com/product/5613>

This is a small passive adapter board. It has an 18-pin EYESPI FPC socket on one side and through-hole pads (breadboard-friendly) on the other, breaking out all 18 EYESPI signals to individual pins. **It is not a microcontroller and cannot run code** — it is purely a wiring adapter.

How it works with a Feather:

[Any Feather]—7 wires—[#5613 breakout]—EYESPI cable—[EYESPI display]

You still need to solder or jumper ~7 wires from the Feather's SPI pins (MOSI, SCK, CS, DC, RST, 3V, GND) to the #5613 breakout's through-hole pads. The breakout then sends those signals through the flat flex cable to the display. This is functionally the same amount of wiring as connecting any standalone SPI breakout display — the advantage is that the flex cable is then available for routing the display to a different physical location more neatly.

EYESPI-compatible displays:

Display	#	Size	Resolution	Driver	URL
2.0" 320×240 IPS TFT EYESPI	5800	2.0"	320×240 colour	ST7789	https://www.adafruit.com/product/5800
1.3" 240×240 IPS TFT EYESPI	5393	1.3"	240×240 colour	ST7789	https://www.adafruit.com/product/5393

EYESPI cables (between #5613 breakout and display):

Cable	#	Length	URL
EYESPI Cable 50 mm	5239	50 mm	https://www.adafruit.com/product/5239
EYESPI Cable 100 mm	5240	100 mm	https://www.adafruit.com/product/5240
EYESPI Cable 200 mm	5241	200 mm	https://www.adafruit.com/product/5241

For most builders, a FeatherWing display or the built-in TFT on #5691 is a simpler choice. The EYESPI path is useful if you specifically want a certain display size/shape and a neat flex-cable run, and you are comfortable soldering 7 wires to the #5613 breakout.

Code: EYESPI displays are not auto-initialised — `board.DISPLAY` is not populated. You must initialise them in `code.py` with `displayio` and the appropriate driver library (same as the standalone breakout displays above).

OLED displays (STEMMA QT plug-and-play, monochrome, no colour)

Display	#	Size	Resolution	Driver	URL
0.96" Monochrome OLED	326	0.96"	128×64	SSD1306	https://www.adafruit.com/product/326
1.3" Monochrome OLED	938	1.3"	128×64	SSD1306	https://www.adafruit.com/product/938

Both connect via STEMMA QT — no soldering. Both use the same SSD1306 driver, the same library, and the same I²C address (0x3C). The v2 code works with either with **no changes at all** — simply swap the physical display.

#326 (0.96") is the v2 default — compact, easy to mount on a tube holder. **#938 (1.3")** is a direct plug-in upgrade — 35% larger, identical wiring and code. A better choice if you want a slightly bigger display without any of the complexity of a TFT.

Wireless Display — ESP-NOW Remote Mirror

Use this option when the **sensor must be behind you** (tube runs behind the wheelchair, head support, or mounting arm) but you need to **see the display in front of you**. A second small board receives the display state wirelessly and shows it on its own screen — no cable runs between them.

How it works: The main board broadcasts the four display fields (group name, Morse buffer, last action, status) over **ESP-NOW** once every 100 ms. ESP-NOW is a fast, connectionless wireless protocol built into every ESP32 chip — it requires no router, no pairing, no configuration. The second board just listens. Range is approximately 30 m indoors.

Each board runs **at least CircuitPython 9** (earlier versions did not include the `espnw` module). Beyond that, the two boards are completely independent — see §12 "Wireless display → CircuitPython versions on the two boards" if you need the full compatibility details.

Option W1 — Second ESP32-S3 Reverse TFT Feather #5691 ★ Recommended

<https://www.adafruit.com/product/5691> — \$34.95

This is the exact same board as the main AeroMorse unit. The built-in 240×135 colour TFT mirrors the main display perfectly — same four rows, same font size, same group colours (blue for Keyboard, green for Mouse, orange for Macro). No extra display hardware to buy or wire.

Advantages	Notes
Identical display to the main board	Each board runs its own CircuitPython independently
No extra display purchase or wiring	See library note above if versions differ
Full colour with group indicator colours	Can run from USB-C charger/power bank or LiPoly battery
No new libraries needed	Does not act as a keyboard (has its own <code>boot.py</code>)

Powering the receiver wirelessly — LiPoly battery options:

The #5691 has a built-in JST-PH battery connector and charges over USB-C. Plug in any of the batteries below and the receiver runs completely cable-free.

Adafruit #	Capacity	Est. runtime	Size / notes	Price	URL
#3898	400 mAh	~3–5 hrs	Slim flat pack — designed to sit flush against the back of a Feather; thinnest option	\$6.95	https://www.adafruit.com/product/3898
#1578	500 mAh	~4–6 hrs	Small pouch, slightly thicker than #3898; good balance of size and runtime for half-day use	\$7.95	https://www.adafruit.com/product/1578
#258	1200 mAh	~9–14 hrs	Full-day runtime; fits in a small case or pouch; recommended for all-day use	\$9.95	https://www.adafruit.com/product/258

Runtime estimates assume ~80–130 mA active draw (ESP32-S3 + TFT on + ESP-NOW receiving). Recharging happens automatically when the USB-C cable is plugged back in — no separate charger needed.

Files needed on the second board: See **\$9.4.1 Option W1** for the complete file tree and step-by-step setup. In short: `receiver.py` → `code.py`, the **same** `boot.py` from the sender (auto-detects the receiver role because no `morse_map.py` is present), and one library folder (`adafruit_display_text/`) under `/lib`. No HID library, no Morse map.

Option W2 — Adafruit MagTag (2.9" e-ink, plug-and-play)

Adafruit MagTag 2025 Edition — 2.9" Grayscale E-Ink WiFi Display #4800 — \$34.95

A 2.9" 296×128 grayscale e-ink display on an ESP32-S2, with four onboard NeoPixels, USB-C, LiPoly charging, and magnet-friendly mounting. The e-ink is much more readable from across a room than the W1 #5691's small TFT, and the screen stays on with zero power draw between refreshes.

⚠ **CircuitPython 10.x required.** The 2025 Edition MagTag (with the SSD1680 e-ink driver) **will NOT work with CircuitPython 9.2.x or earlier**. You must install CircuitPython **10.x.x or later** on the MagTag before `receiver_magtag.py` will run. More information at <https://learn.adafruit.com/adafruit-magtag>.

What you see vs Option W1. E-ink physically cannot refresh fast enough for AeroMorse's 10 Hz pattern preview. The W2 receiver intentionally shows **three** of the four fields broadcast by the main board, dropping the two that change every loop iteration:

Field	Option W1 (TFT)	Option W2 (MagTag e-ink)
Group name ([KEYBOARD])	✓	✓
Live Morse buffer (. - . .)	✓	X (e-ink too slow)
Last action (UP ARROW)	✓	✓
Modifiers / status (SHIFT)	✓	✓
Pressure bar	✓	X (continuous changes)
Group colour indicator	✓ (TFT colour)	✓ (4 onboard NeoPixels)

The four NeoPixels show the **group colour** matching `code.py`'s `_GROUP_COLORS` palette — so even though the e-ink itself is grayscale, you can tell at a glance which mode the device is in from across a room.

Refresh policy. The receiver throttles refreshes to once every ~2 seconds and only redraws when a field has actually changed. A fast burst of typing won't queue 50 refreshes — only the last state at the moment the throttle expires gets drawn. The "last action" line will therefore lag the actual typing by 1–2 s; for live keystroke viewing use Option W1 instead.

When to choose W2 over W1: - Across-the-room readability (e-ink contrast is far better than a small TFT at distance). - The display will sit unused for long periods between updates — e-ink burns no power except during the brief refresh. - A caregiver / family member's glance display where 1–2 s lag is fine. - Sleek aesthetic (the MagTag is designed for wall mounting).

Files needed on the MagTag: see §9.4.1 Option W2 for the complete file tree. In short:

`receiver_magtag.py` → `code.py`, the **same** `boot.py` from the sender (auto-detects receiver role), and the `adafruit_magtag/` library folder (which pulls in several support libraries — listed in the `receiver_magtag.py` file header).

The MagTag is also available with a header strip and other variants; for this build any 2025 Edition MagTag (SSD1680) works.

Wireless display comparison

	Option W1 — Second #5691	Option W2 — MagTag #4800
Cost	\$35	\$35
Display	240×135 colour TFT	2.9" 296×128 e-ink
Refresh rate	100 ms	~2 s
Live Morse preview	✓	✗ (e-ink too slow)
Pressure bar	✓	✗
Group colours	✓ Native	✓ Via 4 NeoPixels
Assembly	Plug USB-C in	Plug USB-C in
Best viewing distance	Arm's length	Across a room
CircuitPython required	9.x+	10.x+
Wireless power	✓ LiPoly	✓ LiPoly
Availability	adafruit.com	adafruit.com

Display driver library required per display chip

Chip	Library	Used by
Built-in (board.DISPLAY)	<i>(none — built in)</i>	#5691 Reverse TFT Feather — uses the repo root <code>code.py</code>
SSD1306	<code>adafruit_displayio_ssd1306</code>	#326 (0.96" OLED) and #938 (1.3" OLED)
ILI9341	<code>adafruit_ili9341</code>	#3315, #1770, #1743
HX8357D	<code>adafruit_hx8357</code>	#3651, #5872, #2050
ST7789	<code>adafruit_st7789</code>	#4311

Note: Adding a larger TFT display requires updating the display initialisation section at the top of `code.py`. Ask for help with this step if needed.

6. Speaker Options

The speaker gives audio feedback — a tone for every dot and dash, and a different tone when a pattern fires. It is optional but strongly recommended: hearing the rhythm helps with timing.

All speaker options connect to the same three Feather signals: A0 (audio), 3V (power — S1/S3 only), and GND. If your Feather has header pins they plug in via breadboard or jumper wires. If your Feather has bare PCB pads, the speaker wires are soldered directly to those pads. See §8D for the full wiring procedure including the no-headers path.

Option S1 — STEMMA Speaker (easiest) ★ Recommended

Adafruit STEMMA Speaker #3885 — \$5.95 <https://www.adafruit.com/product/3885>

- 1 W built-in amplifier, 8 Ω driver, 3-pin JST PH 2 mm socket
- Powered from 3 – 5 V (the Feather's 3V or USB pin both work)
- Loud enough to hear across a quiet room

Cable is sold separately. Order **Adafruit #4046 — JST PH 2 mm 3-Pin Socket to Color-Coded Cable, 200 mm — \$0.95** along with the speaker. <https://www.adafruit.com/product/4046> One end is the JST PH socket that plugs into the speaker board; the other end is three tinned, color-coded wires that go to the Feather. Without this cable, #3885 is just a bare board with a socket on it.

Wiring (#4046 cable to Feather):

Wire (color)	Feather pin
Black (GND)	GND
White (Audio / signal)	A0
Red (VIN)	3V (or USB / 5V)

Option S2 — Passive piezo with detachable 3.5 mm connection (solderless)

A passive piezo wired via a **3.5 mm jack-and-plug pair**, so the piezo end can be unplugged and swapped. Quieter than Option S1, but every connection is screw-terminal — no soldering anywhere. Any speaker that already terminates in a 3.5 mm mono plug also plugs straight into the same #2915 jack with no adapter.

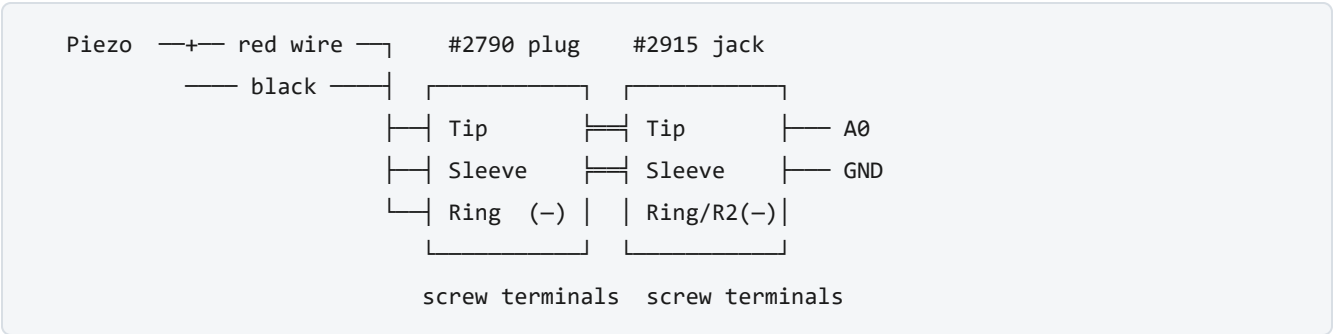
Important: Buy a *passive* piezo, not an *active* one. An active buzzer has a built-in oscillator and will only click, not produce a tone. The two piezos below are confirmed passive and work directly from the Feather's A0 pin via PWM — no resistor, no capacitor, no amplifier (a piezo is a high-impedance capacitive load).

Parts

Qty	Item	Adafruit #	Price	Role
1	TRRS Jack Terminal Block	#2915	\$2.50	Jack on the Feather side (screw terminals)
1	3.5 mm Stereo Audio Plug Terminal Block	#2790	\$2.50	Plug on the piezo side (screw terminals)
1	Small Enclosed Piezo with Leads	#1740	\$0.95	Compact piezo, good for tucking in an enclosure
—	or Large Enclosed Piezo with Leads	#1739	\$0.95	Slightly larger housing, a little louder
2	Female-to-female jumper wires (if Feather has headers)	#266	—	Connect #2915 to Feather A0 / GND

Total: ~\$6 if you already own jumpers.

Wiring



Connection table

Side	Terminal	Connects to	Notes
#2790 plug (piezo side)	Tip	Piezo "+" lead (red)	Polarity match
	Sleeve	Piezo "-" lead (black)	
	Ring	—	Leave empty
#2915 jack (Feather side)	Tip	Feather A0	Audio signal
	Sleeve	Feather GND	Audio return
	Ring	—	Leave empty (mono plug)
	Ring 2	—	Leave empty

Why this works without signal conditioning

The piezo is a capacitive, high-impedance load — direct PWM drive from A0 (square wave at 800–1200 Hz) makes the piezo flex audibly with no need for a series resistor, AC-coupling capacitor, or amplifier. Same electrical path as the bare-lead piezo of older Option S2 builds; the jack-and-plug pair just adds detachability.

Already have a 3.5 mm-plug piezo or speaker?

If your speaker already terminates in a 3.5 mm mono plug, skip the

2790 plug — the existing plug goes straight into the #2915 jack. You

still need the jack itself wired to A0 / GND on the Feather.

Common configurations — how many #2915 jacks you need

Build	#2915 jacks needed	What each one is wired for
Sip-and-puff + on-board STEMMA Speaker (#3885)	0	Sensor uses STEMMA QT; speaker uses JST PH (#4046 cable). No 3.5 mm jack involved.
Sip-and-puff + removable 3.5 mm-plug piezo (Option S2)	1 (Audio)	Tip → A0, Sleeve → GND
AT switches + on-board STEMMA Speaker (#3885)	1 (Switches)	Tip → D5, Ring → D6, Sleeve → GND
AT switches + removable 3.5 mm-plug piezo	2 (Switches + Audio)	Jack 1: Tip → D5, Ring → D6, Sleeve → GND · Jack 2: Tip → A0, Sleeve → GND
AT switches + PAM8302 amp + speaker (Option S3)	1 (Switches)	Tip → D5, Ring → D6, Sleeve → GND. PAM8302 wires to A0/3V/GND directly.

Wiring guide for each role

Switches jack (one #2915, mono or TRS-Y plugs in):

#2915 terminal	Feather pin	Plug type used	What it does
Tip	D5	mono switch <i>or</i> TRS Y-splitter ring 1	Dot input
Ring	D6	TRS Y-splitter ring 2	Dash input
Ring 2	—	(leave empty)	—
Sleeve	GND	mono switch sleeve / TRS sleeve	Ground / switch return

A single mono switch plugged into the same jack gives **dot only** (D5 alone). For both dot and dash, use a TRS Y-splitter (mono-to- stereo) with one switch on each branch.

Audio jack (one #2915, 3.5 mm-plug piezo plugs in):

#2915 terminal	Feather pin	Plug type used	What it does
Tip	A0	piezo's Tip lead	Audio (PWM signal)
Ring	—	(leave empty)	—
Ring 2	—	(leave empty)	—
Sleeve	GND	piezo's Sleeve lead	Audio return

If your piezo doesn't already have a 3.5 mm plug, pair it with the

2790 plug-side terminal block — that gives you a fully solderless

Feather-to-piezo chain (see the parts table at the top of Option S2).

Connecting either jack to the Feather

Two short wires per jack from the screw terminals to the matching Feather pin (or pad, if your Feather has no headers). The §8D speaker assembly already covers the headers-vs-pads choice — same procedure for the switches jack.

For builders running **both** jacks (the AT-switches + audio combination): wire each jack to its set of pins independently. The Tip terminal of the switches jack goes to D5 *only*. The Tip terminal of the audio jack goes to A0 *only*. No shared wires between the two jacks except GND, which both jacks return to.

Option S3 — Small speaker + separate amplifier board

For louder output or a specific speaker size:

- **Adafruit Mono 2.5 W Class D Amplifier PAM8302 #2130** <https://www.adafruit.com/product/2130>
- Any small 4 Ω or 8 Ω speaker (3 W or less)
- Amplifier A+ $\rightarrow$ A0, A- $\rightarrow$ GND, VIN $\rightarrow$ 3V, GND $\rightarrow$ GND
- Speaker connects to the amplifier output terminals

7. Complete Parts Lists

Pick one item from each section. Everything in **Core hardware** is always required.

Core hardware (always required)

Qty	Item	Adafruit #	URL
1	Feather microcontroller (see Section 3)	#5477 recommended	https://www.adafruit.com/product/5477
1	USB-C cable — data + power (not charge-only)	any	—

To confirm a USB cable transfers data: plug it in; if a drive appears on your computer, it is a data cable. Charge-only cables show nothing.

Input method — choose ONE

Sip-and-Puff sensor parts

Qty	Item	Adafruit #	URL
1	LPS33HW Water-Resistant Pressure Sensor — STEMMA QT	#4414	https://www.adafruit.com/product/4414
1	STEMMA QT 4-pin cable 100 mm	#4210	https://www.adafruit.com/product/4210
1 m	Silicone Tubing — 2.5 mm ID, 4.7 mm OD (recommended)	#3659	https://www.adafruit.com/product/3659
—	or Aquarium airline tubing — 3/16" ID, 5/16" OD	—	pet store / hardware store

AT switch — solderless breadboard

Qty	Item	Adafruit #	URL
1	Half-Size Breadboard	#64	https://www.adafruit.com/product/64
1	Breadboard-Friendly 3.5 mm Stereo Jack	#1699	https://www.adafruit.com/product/1699
3	Short male-to-male jumper wires	—	—

Alternative — two jacks, no Y-splitter: Instead of one jack + a Y-splitter, you can use **two #1699 jacks** — one per switch. Each switch plugs directly into its own dedicated jack. This is slightly simpler to wire and removes the Y-splitter from the parts list.

Qty	Item	Adafruit #	URL
1	Half-Size Breadboard	#64	https://www.adafruit.com/product/64
2	Breadboard-Friendly 3.5 mm Stereo Jack	#1699	https://www.adafruit.com/product/1699
4	Short male-to-male jumper wires	—	—

Wiring with two jacks:

Jack	Leg	Feather pin
Jack 1 (dot switch)	TIP	D5
Jack 1 (dot switch)	SLEEVE	GND
Jack 2 (dash switch)	TIP	D6
Jack 2 (dash switch)	SLEEVE	GND

The RING leg on each jack is left unconnected. Both SLEEVE legs can share the same GND pin on the Feather, or use two separate GND holes (the Feather has several GND pins — any of them work).

AT switch — soldered TRRS breakout

Qty	Item	Adafruit #	URL
1	TRRS Jack Breakout Board	#5764	https://www.adafruit.com/product/5764
4	Short wires ~8 cm	—	—

AT switch — #2915 TRRS Terminal Block (no breadboard) ★ Recommended for #5691

Qty	Item	Adafruit #	URL
1	TRRS Jack Terminal Block	#2915	https://www.adafruit.com/product/2915
3	Female-to-female jumper wires (if Feather has header pins)	#266	https://www.adafruit.com/product/266
—	or 3 short wires ~15 cm + soldering iron (if no header pins)	—	—

Display — choose ONE

If you chose the #5691 Reverse TFT Feather, skip this section entirely — the display is already part of that board.

Qty	Item	Adafruit #	URL
1	3.5" TFT FeatherWing Resistive (recommended for table)	#3651	https://www.adafruit.com/product/3651
—	or 3.5" TFT FeatherWing Capacitive	#5872	https://www.adafruit.com/product/5872
—	or 2.4" TFT FeatherWing Resistive	#3315	https://www.adafruit.com/product/3315
—	or 3.5" TFT Breakout (standalone, needs 5 wires)	#2050	https://www.adafruit.com/product/2050
—	or 2.8" TFT Breakout (standalone)	#1770	https://www.adafruit.com/product/1770
—	or 1.3" OLED — STEMMA QT (larger OLED, no code change)	#938	https://www.adafruit.com/product/938
—	or 0.96" OLED — STEMMA QT (v2 code default, compact)	#326	https://www.adafruit.com/product/326

If you choose the OLED #326, you also need:

Qty	Item	Adafruit #	URL
1	STEMMA QT 4-pin cable 400 mm	#5385	https://www.adafruit.com/product/5385

Wireless display — choose ONE (optional, for remote viewing)

Skip this section if you will view the display on the main board. Only needed when the sensor is mounted out of sight and you need a separate display in front of you.

Option W1 — Second Reverse TFT Feather (recommended)

Qty	Item	Adafruit #	URL
1	ESP32-S3 Reverse TFT Feather	#5691	https://www.adafruit.com/product/5691
1	USB-C cable + power bank or charger	—	Powers the display board (if not using a battery)

Optional — run the receiver completely wireless (choose one battery):

Qty	Item	Adafruit #	Est. runtime	URL
1	LiPoly Battery 400 mAh — slim flat pack	#3898	~3–5 hrs	https://www.adafruit.com/product/3898
—	or LiPoly Battery 500 mAh	#1578	~4–6 hrs	https://www.adafruit.com/product/1578
—	or LiPoly Battery 1200 mAh	#258	~9–14 hrs	https://www.adafruit.com/product/258

Option W2 — Adafruit MagTag (e-ink, plug-and-play)

Qty	Item	Adafruit #	URL
1	MagTag 2025 Edition — 2.9" Grayscale E-Ink WiFi Display	#4800	https://www.adafruit.com/product/4800
1	USB-C cable + power bank or charger (or any of the LiPoly batteries listed under W1)	—	—

Speaker — choose ONE (optional but recommended)

Qty	Item	Adafruit #	URL
Option S1	STEMMA Speaker	#3885	https://www.adafruit.com/product/3885
	JST PH 3-pin Socket to Color-Coded Cable, 200 mm (required with #3885)	#4046	https://www.adafruit.com/product/4046
Option S2	TRRS Jack Terminal Block (Feather side)	#2915	https://www.adafruit.com/product/2915
	3.5 mm Stereo Audio Plug Terminal Block (piezo side)	#2790	https://www.adafruit.com/product/2790
	Small Enclosed Piezo with Leads (passive)	#1740	https://www.adafruit.com/product/1740
	or Large Enclosed Piezo with Leads (passive, louder)	#1739	https://www.adafruit.com/product/1739
Option S3	PAM8302 Amplifier + small 8 Ω speaker	#2130	https://www.adafruit.com/product/2130

Tools

Software tools (Thonny etc.) are covered in §9.3 "Install Thonny" — they belong with the software-install steps, not the parts list.

Hardware tools — solderless build (Option B1 breadboard)

If your Feather already has header pins soldered: No special tools needed — the pins press straight into the breadboard.

If your Feather has no header pins (bare holes): You have two options:

Option	What to do	Cost
Get headers soldered	Take the Feather and the included loose header strip to a local makerspace, library maker lab, or electronics repair shop — most will solder headers for free or a few dollars. This is the most reliable long-term solution.	Free–\$5
Order with headers pre-installed	Adafruit sells some Feather boards with headers already soldered. When reordering, look for the listing that says "with headers" in the title.	Same price as bare board

Why not test hook clips? Test hook clips (EZ-hook / Goupchn style) need something cylindrical to hook around — a wire lead, header pin, or IC leg. They cannot reliably grip a flat through-hole pad: there is nothing protruding for the hook to bite, and the hook is too large to engage inside the 0.7–1.0 mm through-hole. They will slide off with the lightest tug. Use one of the two options above instead.

Hardware tools — soldered build (Option B2 TRRS breakout, or soldering headers)

Tool	Notes	Adafruit #	URL
Soldering iron — 40–60 W adjustable	Temperature-controlled irons are easier for beginners	#3685	https://www.adafruit.com/product/3685
Solder — 60/40 rosin-core 0.3 mm	Standard electronics solder; the thin gauge is easier for small pads	#145	https://www.adafruit.com/product/145
Flush cutters	For trimming wire leads flush after soldering	#152	https://www.adafruit.com/product/152
Helping hands	Holds small parts steady while soldering	#291	https://www.adafruit.com/product/291
Wire strippers	For stripping insulation from wire ends	any	hardware store

New to soldering? Adafruit has a free beginner's guide: <https://learn.adafruit.com/adafruit-guide-excellent-soldering>

Useful for any build (optional but handy)

Tool	Purpose	Notes
Digital multimeter	Test switch continuity, check wiring	Any basic model; ~\$10–20 at hardware stores
Small scissors	Cut sip-and-puff tubing to length	Any pair
Silicone tape	Seal tube to sensor if fit is loose	Hardware store; stretchy self-fusing tape
Velcro strips	Mount display or sensor to a surface	Any brand

8. Step-by-Step Assembly

Before you start

Lay out all your parts. Do not plug anything in yet.

The Feather's header pins must be soldered before use if your board came without headers. If your board has header pins already attached, skip this note. Soldering only the headers is straightforward and many local makerspaces or electronics shops will do this for free if you ask.

Step 8A — Connect the display

If you chose a FeatherWing display (#3651, #5872, or #3315)

1. Hold the FeatherWing above the Feather with the display facing up.
2. Line up the two rows of holes on the FeatherWing with the two rows of header pins on the Feather.
3. Press firmly and evenly downward until the FeatherWing sits flush on the Feather. The pins should click through all the way.

That is all — no wires needed.

If you chose the OLED #326

1. Plug one end of the **100 mm STEMMA QT cable** into the STEMMA QT port on the Feather. The connector is keyed and only fits one way — do not force it.
2. Plug the other end into either STEMMA QT port on the **LPS33HW sensor**.
3. Plug one end of the **400 mm STEMMA QT cable** into the remaining port on the LPS33HW.
4. Plug the other end into the STEMMA QT port on the **OLED** (#326).

Chain: **Feather** → **100 mm cable** → **LPS33HW** → **400 mm cable** → **OLED**

If you chose a standalone TFT breakout (#2050, #1770, etc.)

You need 5 short wires. Solder or use header connectors:

Display pin	Feather pin
SCK	SCK
MOSI (or SI)	MOSI
CS	D9
DC	D10
RST	D11
GND	GND
VIN	3V

Pin labels vary slightly between breakout boards. Match by function name, not physical position. Refer to the Adafruit product guide for your specific board.

Step 8B — Connect the pressure sensor (sip-and-puff only)

If you are using AT switches instead, skip this step.

If you also have the OLED the sensor is already in the STEMMA QT chain from Step 8A. No extra wiring needed.

If you are using a TFT FeatherWing or standalone TFT (no OLED in the chain):

1. Plug the **100 mm STEMMA QT cable** into the Feather's STEMMA QT port.
2. Plug the other end into either port on the **LPS33HW sensor**.

The sensor I²C address is 0x5C — it will not conflict with any display.

If your board has no STEMMA QT port (e.g. the Feather nRF52840 Express), wire the sensor to the board's I²C pins instead. Use a **STEMMA QT to male-header cable** ([#4209](#)) or solder wires:

LPS33HW (STEMMA QT wire)	Feather pin
Red — power	3V
Black — ground	GND
Blue — SDA (data)	SDA
Yellow — SCL (clock)	SCL

No config change is needed. From firmware **v1.4**, `code.py` uses `board.STEMMA_I2C()` when the board has a STEMMA QT port and otherwise falls back to `board.I2C()` on the SDA/SCL pins — the serial console prints a note when it does. (Earlier firmware crashed at startup on such boards.)

Attach the sip-and-puff tube:

Push one end of the aquarium airline tubing over the small raised nipple on the top of the LPS33HW board. The fit should be snug. Cut the other end at a comfortable length and place it where you can sip and puff into it naturally.

Step 8C — Connect the AT switches (switch mode only)

If you are using the sip-and-puff sensor instead, skip this step.

Option B1 — Breadboard

See [Appendix D — Breadboard Wiring Walkthrough](#) for the full step-by-step procedure with diagrams.

Option B2 — TRRS breakout (soldered)

1. Solder a short wire to each pad: T (red), R1 (blue), S (black). Leave R2 bare.
2. Connect the other ends:

Wire colour	Feather pin
Red (T)	D5
Blue (R1)	D6
Black (S)	GND

3. Plug AT switches directly into the TRRS jack: - Dot switch: tip-to-sleeve connection = D5 goes low = dot
- Dash switch: ring1-to-sleeve connection = D6 goes low = dash

Option B3 — #2915 TRRS Terminal Block

If your #5691 has header pins (no soldering at all):

1. Cut 3 female-to-female jumper wires to a comfortable length (~15 cm).
2. On the #2915, loosen the Tip, Ring, and Sleeve screws with a small flathead screwdriver.
3. Insert one wire end into each terminal and tighten the screw firmly.
4. Plug the other (female) ends directly onto the Feather header pins: - Tip wire → D5 - Ring wire → D6 - Sleeve wire → GND
5. Plug AT switches into the #2915 jack.

If your #5691 has no header pins (3 solder joints):

1. Cut 3 short wires (~15 cm), strip both ends.
2. Screw one end of each wire into the Tip, Ring, and Sleeve terminals on the #2915.
3. Solder the other ends directly to the D5, D6, and GND pads on the Feather.
4. Plug AT switches into the #2915 jack.

Step 8D — Connect the speaker

All three speaker options end at the same three Feather signals: **A0** (audio), **3V** (power, only for S1 and S3), and **GND**. How those three signals are connected depends entirely on whether your Feather has header pins.

If your Feather has header pins (or sits in a breadboard)

STEMMA Speaker #3885 + #4046 cable — the #4046 cable ends in three tinned bare wires (black / white / red). The tinned tips are stiff enough to push directly into a breadboard, but for the most reliable contact, either (a) solder a short pin or male header onto each wire tip, or (b) crimp/solder each wire into a female jumper that slides onto the Feather's header pin. Match black → GND, white → A0, red → 3V.

#4046 cable wire	Feather pin
Black (GND)	GND
White (Audio / signal)	A0
Red (VIN)	3V

Option S2 piezo (#2915 + #2790 + #1740/#1739) — the piezo's bare leads go into the #2790 plug's screw terminals (red/+ → Tip, black/- → Sleeve). The #2790 plug then plugs into the #2915 jack. On the Feather side, two short F-F jumper wires from the #2915 terminals to header pins: Tip terminal → **A0**, Sleeve terminal → **GND**. No resistor, no cap.

PAM8302 amplifier (#2130) + speaker — same pin mapping as the STEMMMA Speaker (A0, 3V, GND), but soldered to the amp's A+ / VIN / GND pads first. Speaker connects to the amp's two output terminals (polarity not critical).

If your Feather has no header pins (bare PCB pads)

The tinned ends of the #4046 cable have nothing to plug into on a header-less Feather. You have two practical paths — pick one. Both require a small soldering job **on the Feather**, but neither is harder than soldering header pins.

Path 1 — Solder the speaker wires directly to the Feather (recommended)

Cleanest, lowest profile, and the most permanent.

1. Identify the **A0**, **3V**, and **GND** pads on the Feather (printed on the underside).
2. Plug the #4046 cable's JST PH socket end into the #3885 speaker. The other end of #4046 already has three tinned bare wires — no cutting needed. (For the Option S2 piezo build, strip two short wires for the #2915 → Feather link; the piezo itself stays attached to the #2790 plug via screw terminals. For PAM8302, solder the wires to the amp's A+ / VIN / GND pads first.)
3. Trim each tinned wire to ~15 cm if longer than you need, and re-tin the cut ends with a touch of solder.
4. Tin each Feather pad with a touch of solder.

5. Press each tinned wire onto its matching pad and reflow with the iron: - Black (GND) → GND pad - White / Audio → A0 pad - Red (VIN) → 3V pad (#3885 and PAM8302 only — *piezo skips this*)
6. Tug each wire gently to confirm it is fixed.

A 30 W iron, 0.5 mm solder, and a steady hand are plenty. The pads are larger and more forgiving than header-pin holes.

Path 2 — Solder header pins to the Feather first

Use this path if you also want headers for the AT switch jumper wires, or to make the speaker removable later.

1. Solder header pins onto the Feather's A0, 3V, and GND positions only — you don't need a full row of pins, just the three the speaker needs.
2. Then follow the "header pins" instructions above: - #3885 STEMMA Speaker — male pins plug into a breadboard, or use three female-to-female jumpers from the speaker cable to the Feather pins. - Option S2 piezo (#2915 + #2790 + piezo) — two F-F jumpers from the #2915 jack's Tip and Sleeve screw terminals to the Feather's A0 and GND pins. The piezo itself attaches to #2790's screw terminals. - PAM8302 — wire the amp board to A0 / 3V / GND via female jumpers.

A local makerspace, library maker lab, or electronics repair shop will usually solder a few pins for free or a few dollars. See §8 "Before you start" for more on getting headers done if you don't own an iron.

Path 3 — Skip the speaker entirely

The speaker is optional. The OLED / TFT display already shows every dot, dash, group, and last action — many users do not miss the audio at all. Set up the rest of the device first; add the speaker later if you decide you want it.

Quick reference — all paths, all options

Feather has headers?	#3885 + #4046 cable	Option S2 piezo (#2915 + #2790 + #1740/#1739)	PAM8302 + speaker
Yes (breadboard or pins)	Tin/cap 3 wire ends, push into breadboard rows for A0/3V/GND, or use F-F jumpers	2 F-F jumpers from #2915 (Tip / Sleeve) → A0 / GND pins. Piezo wired to #2790 by screw terminals. Zero soldering.	Wire amp to A0/3V/GND via breadboard or F-F jumpers
No (bare pads)	Solder #4046 tinned wires directly to A0/3V/GND pads	Solder 2 wires from #2915 (Tip / Sleeve) screw terminals to Feather A0 / GND pads. Piezo side still screw-terminal.	Solder amp board to A0/3V/GND pads

Step 8E — Set up the wireless display board hardware (optional)

Skip this step if you are not using a wireless display. Software setup for the receiver is covered separately in §9.4.1.

Option W1 — Second Reverse TFT Feather

No hardware assembly needed — the board ships ready to power on. Mount the second Feather face-up where you can see the TFT. Once §9.4.1 is complete it will start mirroring the main board's display within one second of either board powering on.

Power source: any USB-C phone charger, USB power bank, or one of the LiPoly batteries listed in §5 Option W1.

Option W2 — Adafruit MagTag (#4800)

No hardware assembly needed — the MagTag ships ready to power on. Mount it where it will be glanced at from across the room (its e-ink panel is far more readable at distance than the W1 TFT). Once §9.4.1 is complete it will start mirroring the main board's group / last action / status within a few seconds of either board powering on.

Power source: any USB-C phone charger, USB power bank, or one of the LiPoly batteries listed in §5 Option W1 (the MagTag has the same JST-PH 2-pin LiPoly connector and onboard charging as the #5691).

9. Software Installation

Step 9.1 — Install CircuitPython on the Feather

1. Go to <https://circuitpython.org/downloads>
2. Search for your Feather board name (e.g. "ESP32-S3 Feather").
3. Download the latest **stable** `.uf2` file (not a pre-release).
4. Plug the Feather into your computer with the USB-C **data** cable.
5. **Double-tap** the small Reset button on the Feather quickly (two taps within about half a second). - The NeoPixel LED on the Feather turns **green**. - A drive named **FTHRS3BOOT** (or similar) appears on your computer.
6. Drag the `.uf2` file you downloaded onto that drive.
7. The Feather reboots automatically. After a few seconds a drive named **CIRCUITPY** appears. Done.

If **CIRCUITPY** already appears when you plug in (without double-tapping), CircuitPython is already installed — skip straight to Step 9.2.

If you see **FTHRS3BOOT** (or similar) every time you plug in without double-tapping, the Feather has no code loaded — this is normal, continue with Step 9.2.

Step 9.2 — Install the required libraries

1. Go to **<https://circuitpython.org/libraries>**
2. Download the **Bundle** that matches your CircuitPython version. - To find your version: look at the file `boot_out.txt` on the CIRCUITPY drive. It will say something like `Adafruit CircuitPython 10.2.0` — download the bundle that matches your major version number (9.x or 10.x).
3. Open the downloaded `.zip` file. Inside is a folder called `lib`.
4. On the **CIRCUITPY** drive, open (or create) the `lib` folder.
5. Copy these items from the bundle's `lib` folder into CIRCUITPY's `lib` folder:

Always copy these — REQUIRED for every build (including the built-in TFT on #5691 / #5483 / #5300):

```
adafruit_hid/           ← entire folder
adafruit_bus_device/    ← entire folder
adafruit_register/      ← entire folder
adafruit_display_text/  ← entire folder — needed for ALL builds
neopixel.mpy
```

Don't skip `adafruit_display_text/` if you have a built-in TFT. Even though #5691 / #5483 / #5300 don't need a separate display *driver* library (the display is already initialised via `board.DISPLAY`), every build still uses `adafruit_display_text` to render the group name, Morse buffer, last action, and status line. Missing this folder is the most common installation mistake — `code.py` fails immediately at import with `ImportError: no module named 'adafruit_display_text'`.

Copy these if using the sip-and-puff sensor:

```
adafruit_lps35hw.mpy
```

The two libraries that are easy to miss in the bundle — `neopixel.mpy` and `adafruit_lps35hw.mpy` : - `adafruit_lps35hw.mpy` — the tricky one. Your sensor is the **LPS33HW** (#4414), but the CircuitPython driver is named `lps35hw` — the same driver covers both the LPS33HW and LPS35HW, Adafruit just named it after the 35. So don't go looking for an "lps33" file; the correct one is `adafruit_lps35hw.mpy`. Library home page: https://github.com/adafruit/Adafruit_CircuitPython_LPS35HW - `neopixel.mpy` — a **single bare file**, not a folder, and it has **no `adafruit_` prefix**, so it sorts to the bottom of the bundle's `lib/` folder away from everything else — easy to scroll right past. Library home page: https://github.com/adafruit/Adafruit_CircuitPython_NeoPixel

Both ship inside the library bundle `.zip` you already downloaded — look in its `lib/` folder for the exact filenames above. You do not download them separately.

Copy **ONE** of these only if you have an external display:

Display	Additional driver file
OLED #326 or #938	<code>adafruit_displayio_ssd1306.mpy</code>
TFT #3651, #5872, #2050	<code>adafruit_hx8357.mpy</code>
TFT #3315, #1770, #1743	<code>adafruit_ili9341.mpy</code>
TFT #4311	<code>adafruit_st7789.mpy</code>
Built-in TFT (#5691 / #5483 / #5300)	<i>no extra driver — uses <code>board.DISPLAY</code></i>

You only need the items listed above — you do not need to copy the entire bundle, which is very large.

Step 9.3 — Install Thonny and watch the serial console

Before copying the AeroMorse project files in §9.4, install **Thonny** — a free, beginner-friendly code editor that doubles as a serial console. The serial console is how the Feather reports errors when it tries to load `code.py`, the libraries, or `morse_map.py`. **Without watching the console, a problem in §9.4 looks like "nothing happens" instead of a specific error message you can act on.**

Step 9.3.1 — Download and install Thonny

1. Go to <https://thonny.org> and click the download button for your operating system (Windows, macOS, or Linux).
2. Run the installer. No special options are needed — accept defaults.
3. Launch Thonny after installation completes.

Mu Editor was previously recommended by Adafruit but has announced end-of-life in 2026. Thonny is the current recommended alternative.

Step 9.3.2 — Confirm the Feather is plugged in

The Feather should still be connected from §9.1 / §9.2. The **CIRCUITPY** drive must be visible in File Explorer (Windows) or Finder (Mac). If it isn't, unplug and replug the USB-C cable.

Step 9.3.3 — Point Thonny at CircuitPython

1. In Thonny, click **Run** → **Configure interpreter...** (or click the interpreter name shown at the bottom-right of the Thonny window).
2. In the top dropdown, select **CircuitPython (generic)**.

3. In the **Port** dropdown, select the port your Feather is on: - **Windows:** a COM port, e.g. `COM3` or `COM7` — try each if unsure - **macOS:** something like `/dev/cu.usbmodem14101` - **Linux:** something like `/dev/ttyACM0`
4. Click **OK**.
5. The bottom **Shell** panel should now show `>>>` — this is the REPL prompt, confirming Thonny is talking to the Feather.

Can't find the right port? Windows: open Device Manager → Ports (COM & LPT) — the Feather appears as "USB Serial Device" or "CircuitPython CDC Control". Mac: open Terminal and type `ls /dev/cu.*` — look for `usbmodem`.

Step 9.3.4 — Leave the Shell panel visible

The Shell panel at the bottom of the Thonny window will display everything `code.py` prints (errors, calibration messages, debug output). Keep this panel visible while you complete §9.4. If a file copy fails, or a library is missing, the Shell will print an exact error message naming the file and line number — much easier to act on than guessing.

A healthy startup after §9.4 looks like this in the Shell:

```
Calibrating – do not sip or puff ...
Baseline: 1013.241 sip<1008.241 puff>1018.241
Calibration complete – ready for input.
```

If you see a red `Traceback (most recent call last):` line instead, the last line of the traceback names the problem (e.g. `ImportError: no module named 'adafruit_lps35hw'` — means that library file is missing from `/lib`; re-do §9.2). Note: in some Thonny versions tracebacks render in plain text rather than red.

Step 9.3.5 — Capturing a fresh boot trace (Ctrl+D)

The Shell panel only shows messages that fire *while Thonny is listening*. The first time you connect, the device has usually already booted — the `Calibrating...` and `Baseline:` lines you want to see have already scrolled past. You don't need to physically replug the cable or hit a reset button to recapture them.

Click in the Shell panel, then press Ctrl+D. That sends a CircuitPython "soft reset" — the device re-runs `boot.py` and `code.py` from scratch and prints everything to the Shell as it happens. Use it any time you:

- want to see the boot output again,
- have edited `code.py` and want a clean restart (saving normally triggers an auto-reload, but Ctrl+D gives you a confirmed clean run),
- need to capture a traceback for diagnosing a problem.

A healthy Ctrl+D in sensor mode looks like:

```
soft reboot
Auto-reload is on. ...
code.py output:
Calibrating – do not sip or puff ...
Baseline: 1013.241 sip<1008.241 puff>1018.241
Input mode: 2-switch (1-sw input = dot, 3-sw accept = long_dash)
Code repeat: off long-press cycles group: True
ESP-NOW: disabled by USE_WIRELESS_DISPLAY = False
```

If `ESP-NOW: init failed (ESP-NOW error 0x3065)` shows up, that's a known CircuitPython 9.0.3 bug — see §12 "Wireless display" for the workaround.

Useful Thonny shortcuts

Action	Shortcut
Save file (and trigger Feather restart)	Ctrl+S / Cmd+S
Stop running program / go to REPL	Ctrl+C (in Shell)
Restart program from REPL	Ctrl+D (in Shell)
Open file from CIRCUITPY drive	File → Open → CircuitPython device

Further Thonny details (REPL examples, alternative editors like VS Code) are in `TOOLS_AND_GUIDES.md`.

Step 9.4 — Copy the AeroMorse project files

Step 9.4.0 — Download the AeroMorse software (do this first)

The AeroMorse firmware (`code.py` , `boot.py` , `morse_map.py` , `config.py` and the wireless-receiver files) is **not** part of the Adafruit library bundle you downloaded in §9.2. It lives in one place — the official GitHub repository:

👉 <https://github.com/jlubin2001/AeroMorse>

Always download from that repo. Copies posted elsewhere on the web may be older or modified versions; the repo is the single source of the latest, correct files.

To download (no Git account or software needed):

1. Open <https://github.com/jlubin2001/AeroMorse> in a browser.

2. Click the green `< > Code` button, then **Download ZIP**.
3. Unzip the downloaded file. Everything you copy in the steps below comes from the **root** of that unzipped folder.

Which version do I have? Every AeroMorse `.py` file has a version and release date in its header comment near the top — e.g. `AeroMorse code.py – version 1.22 (released 2026-10-03)`. Open the file in Thonny (or any text editor) to check. If a file you found somewhere else has no such header, or an older date than the repo, replace it with the repo copy. Keep `code.py`, `boot.py`, `morse_map.py` and `config.py` all at the **same** version. The wireless-display programs (`receiver.py`, `receiver_magtag.py`) share the same version number, so every AeroMorse file from one release says the same version. The main device's startup screen shows its version (e.g. `v1.9`) until the first sip, puff or switch press.

Filenames must be exact. CircuitPython looks for `code.py`, `boot.py`, `morse_map.py`, and `config.py` by *literal* name. If you copy a file as `morse_map_darci.py` (or any other variant name) it will be ignored — `code.py` does `from morse_map import groups`, and that import resolves only to a file named `morse_map.py`. **Rename the file to `morse_map.py` before or after copying** — see the Darci callout below.

The repo root `code.py` is the active firmware — it targets boards with a built-in 240×135 colour TFT (#5691 Reverse TFT, #5483, #5300) and also drives an EYESPI or FeatherWing TFT with a one-line display init swap (see §5).

Copy these four files from the **repo root** to the root of the CIRCUITPY drive (not inside any subfolder):

```
boot.py
code.py
config.py
morse_map.py
```

`config.py` holds every user-tunable setting — sensor thresholds, switch mode, code repeat, audio pitches, etc. You edit `config.py` to change behaviour, not `code.py`. See §10 for the full settings reference.

If you are a Darci USB user starting from `morse_map_darci.py`: Copy `morse_map_darci.py` to the CIRCUITPY drive, then **rename it to `morse_map.py`** (right-click → Rename on Windows/Mac, or `mv` on Linux). Do not leave both files on the drive — CircuitPython will only load the one literally named `morse_map.py`, and a stale extra file just wastes flash.

The CIRCUITPY drive should now look like this:

```

CIRCUITPY/
├─ boot.py
├─ code.py
├─ config.py          (user-tunable settings – see §10)
├─ morse_map.py
└─ lib/
    ├─ adafruit_hid/          (always – keyboard/mouse HID)
    ├─ adafruit_bus_device/   (always – required by sensor + display libs)
    ├─ adafruit_register/     (always – required by sensor + display libs)
    ├─ adafruit_display_text/ (always – text labels on the screen)
    ├─ neopixel.mpy           (always – onboard NeoPixel)
    ├─ adafruit_lps35hw.mpy    (only if using sip-and-puff sensor)
    ├─ adafruit_displayio_ssd1306.mpy (only if using OLED #326 or #938)
    ├─ adafruit_hx8357.mpy    (only if using TFT #3651, #5872, or #2050)
    ├─ adafruit_ili9341.mpy    (only if using TFT #3315, #1770, or #1743)
    └─ adafruit_st7789.mpy    (only if using TFT #4311)

```

Built-in TFT boards (#5691 / #5483 / #5300) get nothing extra from the per-display list — `board.DISPLAY` is initialised by the firmware, so no driver `.mpy` is needed. They still need every "always" entry above.

Safely eject the CIRCUITPY drive before unplugging. The Feather reboots and runs the code automatically.

The first boot after copying `boot.py` takes a few extra seconds. This is normal — the board is negotiating USB HID with the host. If nothing seems to happen, unplug and replug the USB cable once.

Step 9.4.1 — Software for the Wireless Display receiver (optional)

Skip this step if you are not using the ESP-NOW wireless display (\$5 "Wireless Display — ESP-NOW Remote Mirror"). The main board you set up above is already fully functional.

The receiver is a **second** board that runs entirely independently of the main board — its own CircuitPython firmware, its own `code.py`, its own `/lib` folder. It listens for ESP-NOW broadcasts from the main board and mirrors the four display fields on its own screen. It does **not** act as a keyboard or mouse; you do not load `morse_map.py` or `adafruit_hid` on it.

Files on the receiver

File on your computer	Copy to the receiver as
<code>boot.py</code>	<code>boot.py</code> (same file used on the sender)
<code>receiver.py</code> (W1) or <code>receiver_magtag.py</code> (W2)	<code>code.py</code> (rename when copying)
<code>receiver_config.py</code>	<code>receiver_config.py</code> — the display's own settings: wireless channel, screen brightness and rotation, no-signal / sleep / auto-restart times. Optional (defaults are built in), but copy it so you can change them. The main device does not use it

One `boot.py` for every board. The same `boot.py` from the sender works unchanged on every receiver — it inspects the filesystem at boot and chooses its role automatically: - If `morse_map.py` is present → **sender**: enables USB HID (Keyboard, Mouse, ConsumerControl). - If `morse_map.py` is absent → **receiver**: leaves HID off so the host doesn't see the second board as a duplicate keyboard. - If `morse_map.py` is absent AND an empty `/hide` file is also present → **hidden receiver**: also disables the CIRCUITPY drive and serial port so the host sees nothing at all. Useful once the receiver is configured and you want a clean USB enumeration.

Do NOT copy `morse_map.py` to the receiver. It's the file the boot detection uses to choose sender vs receiver — and the receiver doesn't decode any Morse anyway (it just renders what the main board sends).

Firmware on the receiver (Step 9.1 equivalent)

Install CircuitPython on the receiver the same way you did on the main board in §9.1. The receiver needs **at least CircuitPython 9** so the `espnow` module is included. The main board and the receiver do **not** need to be on the same CircuitPython major version — see §12 "Wireless display → CircuitPython versions on the two boards" for the details.

Option W1 — Second ESP32-S3 Reverse TFT Feather #5691 (recommended)

```
CIRCUITPY/    (on the W1 receiver)
├─ boot.py      (= same boot.py as the sender)
├─ code.py      (= renamed copy of receiver.py)
└─ lib/
    └─ adafruit_display_text/    (only library needed)
```

Just **two files** at the root plus **one library folder** under `/lib`. Nothing else. The built-in TFT is initialised via `board.DISPLAY` so no display-driver `.mpy` is needed.

`adafruit_hid/`, `adafruit_bus_device/`, `adafruit_register/`, `neopixel.mpy`, and `adafruit_lps35hw.mpy` are **not** required on the receiver — leaving them off saves flash and avoids confusion.

Option W2 — Adafruit MagTag (#4800, 2.9" e-ink)

⚠ **CircuitPython 10.x required.** The 2025 Edition MagTag will NOT work with CircuitPython 9.2.x or earlier — see §5 Option W2 and the warning banner at the top of `receiver_magtag.py`.

The W2 receiver file is `receiver_magtag.py` (a different file from the W1 `receiver.py` because the display layout and refresh model are fundamentally different — fewer fields, slower throttled refresh).

```
CIRCUITPY/    (on the W2 MagTag receiver)
├─ boot.py          (= same boot.py as the sender)
├─ code.py          (= renamed copy of receiver_magtag.py)
├─ lib/
│   ├─ adafruit_magtag/      (e-ink + peripherals)
│   ├─ adafruit_display_text/
│   ├─ adafruit_bitmap_font/
│   ├─ adafruit_io/          (pulled in by adafruit_magtag)
│   ├─ adafruit_portalbase/   (pulled in by adafruit_magtag)
│   ├─ adafruit_fakerequests.mpy (pulled in by adafruit_magtag)
│   ├─ adafruit_lis3dh.mpy    (onboard accelerometer driver)
│   ├─ neopixel.mpy
│   └─ simpleio.mpy
```

Most of the libraries are pulled in transitively by `adafruit_magtag/`; copy them all from the CircuitPython 10.x library bundle. The `receiver_magtag.py` file header lists the exact required set.

Verifying the receiver

1. Plug the receiver into a USB-C power source.
2. Within ~1 second of the **main board** booting, the receiver's display should mirror the four display fields (group, Morse buffer, last action, status).
3. If it stays blank or shows "No signal", open the receiver in Thonny and check the Shell — `ImportError` messages name the missing file. See §12 "Wireless display" for the full diagnostic table.

10. Configuration

All user-tunable settings live in `config.py` at the root of the CIRCUITPY drive (not in `code.py`). Open `config.py` in Thonny (§9.3) or any plain-text editor — one assignment per line, grouped into nine sub-sections (Input, Input mode, Strong sip/puff, Timing, Code repeat, Audio, Mouse, Repeat exclusions, Display / wireless). Each line has a short trailing hint; for **full per-setting explanations including when and how to tune**, see [Appendix E — Configuration Reference](#) at the end of this guide.

Save in Thonny → the Feather auto-reloads with the new values. Press **Ctrl+D** in the Shell panel any time to force a fresh restart with full boot output (§9.3.5).

Key settings — quick reference

One row per setting with the value shipped in `config.py` and a one-line hint. Tables below follow the same order and grouping as the file. For the full "what does this do / when do I change it / how does it interact with other settings" explanation, jump to Appendix E.

Device name

Setting	Shipped	Hint
<code>DEVICE_NAME</code>	<code>"AeroMorse"</code>	Name the computer shows for this device (e.g. Windows <i>Bluetooth & devices</i>) instead of "Feather ESP32-S3 Reverse TFT" — e.g. <code>"AeroMorse Green"</code> . Up to 40 plain characters, in quotes. Applied by <code>boot.py</code> after an unplug/replug ; if Windows still shows the old name, remove the device there and replug

Input — sensor / switches / thresholds

Setting	Shipped	Hint
USE_SENSOR	True	False = AT switches on D5/D6 instead of LPS33HW sensor
THRESH_SIP	2	hPa below baseline = dot (raise if false triggers)
THRESH_PUFF	2	hPa above baseline = dash
DEBOUNCE_SAMPLES	3	Consecutive agreeing samples to confirm a state change (~13 ms each)
POINTS_TO_AVERAGE	8	Reserved — not used in the threshold path; changing it does nothing
SENSOR_FILTER_ENABLED	True	LPS33HW hardware low-pass. False = lowest latency, noisier
SENSOR_FILTER_HEAVY	True	True = ODR/20 (~40–60 ms lag), False = ODR/9 (~half). Major typing-speed lever
BASELINE_DRIFT_S	30	Auto-zero time constant (s) — baseline tracks ambient pressure drift while idle. 0 disables
REPEAT_SPLIT_PCT	0	Split on a dip: a sip/puff that drops below this % of its peak and then climbs again counts as two. Fixes two quick sips/puffs running together (p → r, space → w). Try 60. 0 = off
REPEAT_SPLIT_RISE	1.0	hPa the pressure must climb again after the dip before it is split. Keep above sensor noise
DIAG_LOG_S	0	Diagnostics (v1.19+): every this many seconds one DIAG ... line of timing figures (loop speed, longest blind moment, screen / wireless / key-send time, sensor reading rate, shortest sip/puff and rest) goes to the USB serial log. The first / worst / last line are saved on devicereset, and automatically every 5 minutes while you type (so they survive an unplug); the next run prints them as DIAG PREV ... (and the run before that as DIAG PREV2 ...). For tracking down "types badly until restarted". 0 = off; try 10
DOT_PIN	board.D5	Switch mode only — GPIO for the dot switch
DASH_PIN	board.D6	Switch mode only — GPIO for the dash switch

Input mode — 1 / 2 / 3 switch

Setting	Shipped	Hint
SWITCH_MODE	2	1 = single-switch timed, 2 = paddle, 3 = paddle + explicit Accept
ONE_SWITCH_INPUT	"dot"	Mode 1 only — "dot" or "dash"
ONE_SWITCH_DOT_MS	200	Mode 1 only — press ≤ this ms = dot, longer = dash
THIRD_SWITCH_GESTURE	"long_dash"	Mode 3 only — "long_dash" or "long_dot" = Accept

Strong sip / strong puff

Setting	Shipped	Hint
STRONG_SIP_ACTION	"group 2"	Action fired by a strong sip; "" = disabled
STRONG_PUFF_ACTION	"group 1"	Action fired by a strong puff; "" = disabled
THRESH_SIP_STRONG	15	Sensor mode — hPa for strong-sip detection
THRESH_PUFF_STRONG	15	Sensor mode — hPa for strong-puff detection
STRONG_OFF_IN_GROUPS	(4,)	Groups where strong sip/puff is off — a hard sip/puff there is just a normal dot/dash. Default Scanning (4), so hard presses can't knock you out of Switch Control. () = on everywhere
SWITCH_GROUP	7	Group that acts as two plain switches (no Morse): sip holds SWITCH_SIP_KEY, puff holds SWITCH_PUFF_KEY for as long as you keep going. 0 = no Switch group. See AEROMORSE_SWITCH_MODE_GUIDE.md
SWITCH_SIP_KEY	"ENTER"	Key held while sipping in the Switch group (a Keycode name)
SWITCH_PUFF_KEY	"SPACE"	Key held while puffing in the Switch group
SWITCH_IDLE_EXIT_S	20	Seconds with no sip/puff before the Switch group returns to SWITCH_EXIT_GROUP by itself. 0 = never
SWITCH_EXIT_PUFF_S	5.0	Or hold one puff this many seconds to leave the Switch group
SWITCH_EXIT_GROUP	1	Group you return to when leaving the Switch group (1 = Keyboard)
SWITCH_COUNTDOWN_S	10	(v1.21+) For the last this-many seconds before the Switch group is left the screen counts down, e.g. MOUSE IN 5 — both for the idle exit and while holding the long exit puff. Any sip/puff clears it. 0 = no countdown

Timing

Setting	Shipped	Hint
ACCEPT_DELAY	0.3	Idle seconds before a pattern commits. Lower (0.2) = faster; sip-and-puff users often land 0.3 – 0.7
MOUSE_ACCEPT_DELAY	0.15	Same as ACCEPT_DELAY but only in Group 2 (mouse) , so clicks fire sooner while keyboard typing keeps the longer, safer ACCEPT_DELAY. Must stay above the gap <i>between the symbols of a pattern</i> or mouse patterns split. Lower = snappier clicks. Delete the line to disable (mouse then uses ACCEPT_DELAY).
LONG_PRESS	1.0	Seconds to hold for cycle / Accept gesture

Code repeat (Darci-style hold-to-repeat)

Setting	Shipped	Hint
<code>CODE_REPEAT</code>	False	<code>True</code> = Darci-style hold-to-repeat (mode 2 only)
<code>DOT_REPEAT_MS</code>	200	ms per auto-repeated dot
<code>DASH_REPEAT_MS</code>	600	ms per auto-repeated dash
<code>CODE_REPEAT_MAX</code>	8	Cap on symbols per held stream
<code>LONG_PRESS_CYCLES_GROUP</code>	True	<code>False</code> disables long-press group cycling

Audio — speaker pitches (Hz) and blip durations (s)

Setting	Shipped	Hint
<code>AUDIO_PIN</code>	<code>board.A0</code>	GPIO the speaker is on
<code>BEEP_DOT_FREQ</code>	1200	Hz — dot (sip) sidetone
<code>BEEP_DASH_FREQ</code>	800	Hz — dash (puff) sidetone
<code>CONFIRM_FREQ</code>	1050	Hz — short blip when an action fires
<code>GROUP_FREQ</code>	550	Hz — short blip on group change
<code>BEEP_CONFIRM_S</code>	0.06	Confirm-blip duration (s)
<code>BEEP_GROUP_S</code>	0.14	Group-change blip duration (s)

Mouse — Group 2 movement, speed, clicks

Setting	Shipped	Hint
<code>MOUSE_SPEED_NORMAL</code>	2	Default speed multiplier in Group 2
<code>MOUSE_SPEED_SLOW</code>	1	Speed after <code>mslow</code>
<code>MOUSE_SPEED_FAST</code>	3	Speed after <code>mfast</code>
<code>MOUSE_SPEED_FACTOR</code>	2	Overall scale on all mouse moves
<code>MOUSE_REPEAT_DELAY</code>	0.040	Seconds between repeat ticks (40 ms)
<code>MOUSE_CLICK_MOD_DELAY</code>	0.030	Seconds to settle an armed modifier before/after a click (Ctrl+click reliability)
<code>MOUSE_CLICK_KEEPS_MODS</code>	True	<code>True</code> = modifier survives a click (Ctrl+click multi-select); <code>False</code> = one-shot
<code>MOUSE_CLICK_HOLD</code>	0.060	Seconds the button is held per click — a zero-length click is ignored by Windows scrollbar tracks
<code>MOUSE_CLICK_GAP</code>	0.040	Seconds between the two clicks of a double-click

Repeat exclusions

Setting	Shipped	Hint
<code>NO_REPEAT_KEYS</code>	<code>("PAGE_UP", "PAGE_DOWN")</code>	Keys (by <code>Keycode</code> name) that never auto-repeat. Add e.g. <code>"HOME"</code> , <code>"END"</code> . A combination joined with <code>+</code> (<code>"ALT+TAB"</code> , v1.20+) blocks exactly that combination. Arrows deliberately omitted
<code>SECRETS_AUTOLOCK_MIN</code>	<code>0</code>	Only matters with a PIN-locked <code>macro_secrets.enc</code> (see README <i>PIN-locked secrets</i>). Minutes with no sip/puff before unlocked secrets lock again. <code>0</code> = stay unlocked until power-off or Macro <code>L</code> (<code>lock</code>). Secrets are always locked at power-up

Display / wireless

Setting	Shipped	Hint
<code>USE_DISPLAY</code>	<code>True</code>	<code>True</code> = this board has a built-in screen (the default #5691 Reverse TFT). Set <code>False</code> on a board with no screen (e.g. a screenless ESP-NOW sender) — the device still types over USB and still broadcasts to a wireless receiver; only the local screen is skipped. A missing screen is also auto-detected, so a screenless board won't crash even if this is left <code>True</code> . Also set <code>False</code> on a board that has a screen when you only watch the wireless display (v1.22+): the built-in screen is blanked and its backlight switched off, removing screen-drawing pauses of 60–100 ms that can swallow a quick sip or puff (the backlight comes back on if the program stops with an error)
<code>DISPLAY_BRIGHTNESS</code>	<code>1.0</code>	Screen backlight <code>0.1</code> (dim) – <code>1.0</code> (full); applies on save; never below <code>0.1</code>
<code>DISPLAY_ROTATION</code>	<code>0</code>	<code>0</code> / <code>90</code> / <code>180</code> / <code>270</code>
<code>USE_WIRELESS_DISPLAY</code>	<code>False</code>	<code>True</code> = ESP-NOW broadcast to a wireless receiver (adds ~80–100 mA). Leave <code>False</code> unless you have a receiver
<code>ESPNOW_CHANNEL</code>	<code>1</code>	2.4 GHz channel (1–13). Must match <code>ESPNOW_CHANNEL</code> in the display's <code>receiver_config.py</code>

Input modes — what `SWITCH_MODE` does

Mode	Behaviour	Group cycling
1	Only <code>ONE_SWITCH_INPUT</code> is active. Press $\leq$ <code>ONE_SWITCH_DOT_MS</code> (200 ms) = dot, longer = dash. Pause <code>ACCEPT_DELAY</code> = end-of-letter.	Long-press ($\geq$ <code>LONG_PRESS</code>) cycles forward. No backward cycle — use a g0 Morse pattern.
2 (default)	Dot input = dot, dash input = dash. Pause <code>ACCEPT_DELAY</code> = end-of-letter.	Long-press dot = cycle back; long-press dash = cycle forward.
3	Dot input = dot, dash input = dash. Explicit Accept: long-press of <code>THIRD_SWITCH_GESTURE</code> commits immediately.	Long-press of the <i>other</i> gesture cycles forward. No backward cycle — use a g0 Morse pattern.

For a side-by-side comparison of how AeroMorse, Darci USB, morAce, and Adap2U each implement 1/2/3-switch input, see `MORSE_DEVICES_COMPARISON.md`.

Save the file to the CIRCUITPY drive. The Feather reloads automatically within a few seconds.

11. First Power-On Test

1. Plug the Feather into your computer.
2. Wait 3–5 seconds. You should see: - The display showing **[Keyboard]** - If using the sensor: the message "Calibrating..." in the serial console - A short beep from the speaker when calibration finishes (sensor mode only)
3. Open a plain-text editor on your computer (Notepad, TextEdit).
4. Click inside the editor so it is focused.

Quick test — sensor mode

Gently sip (dot) · pause · gently puff (dash) · pause · puff (dash). Wait 0.3 seconds. The letter **W** (`.-.-`) should appear in the text editor and the speaker should have beeped three times.

Quick test — switch mode

Press dot switch · dot switch · dash switch (`. . -`). Wait 0.3 seconds. The letter **U** (`..-`) should appear.

If nothing appears

- Make sure the text editor window is focused (click in it).
- Unplug and replug the USB cable — the first connection after installing `boot.py` sometimes needs a fresh plug.
- In sensor mode: make sure the tube is pushed onto the sensor and you are breathing into it, not blowing from the side.

- In switch mode: confirm `USE_SENSOR = False` is set in `code.py`.
-

12. Troubleshooting

Grouped by hardware option, in the same order as §4 – §6 (Input → Display → Speaker → Wireless display → Software / general).

Input — sip-and-puff sensor (#4414)

"Calibrating" message hangs at startup - The LPS33HW was not found. Check the STEMMA QT cable between the Feather and the sensor — both connectors must click in firmly. - Confirm `adafruit_lps35hw.mpy` is in `/lib` on the CIRCUITPY drive.

False pressure triggers (random dots/dashes while not sipping or puffing) - Raise `THRESH_SIP` and `THRESH_PUFF` to 3 or 5 in `config.py` if it happens immediately on touching the tube. Combined with a low `DEBOUNCE_SAMPLES` change. - Check the sip-and-puff tube is not kinked or pinched. - Confirm auto-zero is on: `BASELINE_DRIFT_S` should be non-zero (default 30). If it's 0, the boot-time baseline is fixed and will go stale over minutes/hours.

Pressure bar slowly grows red/green over time, then fires random codes - Classic symptom of **ambient pressure drift** (weather, HVAC, temperature). The LPS33HW measures absolute atmospheric pressure, so a slow real-world change in the room shows up as a slow drift away from the boot-time baseline — eventually crossing the threshold and firing as a phantom sip or puff. - Confirm `BASELINE_DRIFT_S` in `config.py` is non-zero (default 30). If it's 0, auto-zero is disabled — set it to 30 and reboot (Ctrl+D in Thonny). The pressure bar should stay near zero even after the device sits idle for hours. - If even with auto-zero on the bar still drifts visibly, lower `BASELINE_DRIFT_S` to 15 to track faster. Don't go below ~10 or slow deliberate sips may start getting absorbed into the baseline. - See Appendix E " `BASELINE_DRIFT_S` " for the full tuning rationale.

Input — AT switches

AT switches not registering - Confirm `USE_SENSOR = False` in `code.py`. - Test the switch with a multimeter in continuity mode: pressing the switch should beep, confirming Tip connects to Sleeve. - Confirm D5 (dot) and D6 (dash) wires are connected to the correct jack pins.

Input — timing

Pattern fires too early (cuts off long patterns) - Raise `ACCEPT_DELAY` (default 0.3) to 0.4 or 0.5 in `config.py`.

Groups cycle when you did not mean to - Raise `LONG_PRESS` from `1.0` to `1.5` or `2.0` to require a longer hold.

Typing feels slower than my previous Morse device

Experienced Morse users migrating from a switch-based device (Darci USB, morAce, a homebrew paddle) often find they have to deliberately slow down on AeroMorse or the error rate climbs. This is a **tuning** problem, not a limitation — the defaults are deliberately conservative, set for a first-time user who benefits from generous timing. A fluent user should retune.

Understand the failure mode first, because it tells you which knob to turn. With `SWITCH_MODE = 2` a character commits after `ACCEPT_DELAY` of idle. So:

- **Letters running together** (you meant `E T`, you got `A`) means you started the next character before `ACCEPT_DELAY` elapsed. The timeout is too long *for your speed*. **Lower** `ACCEPT_DELAY`.
- **Letters splitting apart** (you meant `C`, you got `N` then `T`) means you paused mid-character longer than `ACCEPT_DELAY`. **Raise** `ACCEPT_DELAY`.

Being forced to slow down is the first case. Work through these in order — the first two matter most:

Setting	Default	Try	Effect
<code>ACCEPT_DELAY</code>	<code>0.3</code>	<code>0.20</code> , then <code>0.15</code>	Directly sets the pause you must leave between characters. The single biggest throughput lever.
<code>SENSOR_FILTER_HEAVY</code>	<code>True</code>	<code>False</code>	Halves the sensor's hardware low-pass lag (~40–60 ms → ~20–30 ms) on every press and release.
<code>DEBOUNCE_SAMPLES</code>	<code>3</code>	<code>2</code>	Saves ~13 ms confirming each edge (~27 ms per element). Don't go to <code>1</code> unless you have a very clean signal.
<code>THRESH_SIP</code> / <code>THRESH_PUFF</code>	<code>2</code>	<code>2</code> , or lower	Lower thresholds trigger earlier in the breath. Only lower these if you are <i>not</i> getting false triggers; use <code>test_pressure.py</code> to check your headroom.
<code>SENSOR_FILTER_ENABLED</code>	<code>True</code>	<code>False</code>	Last resort. Removes hardware smoothing entirely — lowest possible latency, but you will likely need to raise <code>THRESH_*</code> to stop false triggers.

Change **one setting at a time** and type a familiar paragraph after each. Latency changes are easy to misjudge by feel, and stacking three edits at once makes it impossible to tell which one helped.

`POINTS_TO_AVERAGE` **is not a speed knob**. It looks like the obvious thing to lower, but it is not currently used in the threshold path (see Appendix E) — changing it does nothing. Smoothing lives in `SENSOR_FILTER_*` and `DEBOUNCE_SAMPLES`.

Repeated elements collapse — `--` reads as `-`, `..` reads as `.`

A distinct fault with a distinct fix. The signature: the wrong character is always the intended one with a **run of identical elements shortened by one**, never a wrong element type and never a dropped isolated element. Examples from the default map:

Intended	Pattern	Received	Pattern	Collapse
c	- - - .	g	- - .	3 dashes → 2
space	. . - -	w	. - -	2 dots → 1
LEFT_SHIFT	- - . . . -	+	- . . . -	2 dashes → 1

Cause. To register two dots in a row the firmware must see `IDLE → DIT → IDLE → DIT`. Pressure has to fall back inside the idle band *between* the elements and stay there long enough for `DEBOUNCE_SAMPLES` to confirm it. Two things eat that brief dip:

- the sensor's hardware low-pass, which at `SENSOR_FILTER_HEAVY = True` adds ~40–60 ms of group delay and smooths a short return-to-neutral away entirely, and
- `DEBOUNCE_SAMPLES`, which needs that many consecutive agreeing samples (~13 ms each) before it will believe the IDLE.

Together they need roughly 80–100 ms of clean gap between same-type elements. Type faster and your inter-element gaps shrink below that, so the pair merges into one longer element.

This is why the fault often **appears right after you lower** `ACCEPT_DELAY` — nothing is wrong with the new value, you simply started keying faster and crossed the threshold where the filter can no longer resolve adjacent elements.

Fix, in order:

0. (v1.18+) `REPEAT_SPLIT_PCT = 60` — "split on a dip". A sip/puff that drops below 60 % of its peak and then climbs again by `REPEAT_SPLIT_RISE` hPa is counted as two, even if the pressure never got back inside the idle band. It leaves the triggers, filter and timing alone, so try it first if your other settings already suit you. `0` switches it off.
1. `SENSOR_FILTER_HEAVY = False` — halves the group delay so the dip between elements survives. Usually sufficient on its own.
2. `DEBOUNCE_SAMPLES = 3 → 2` — fewer samples needed to confirm the brief IDLE.
3. Steps 1 and 2 are usually enough on their own. Only if they are not, consider raising `THRESH_SIP` / `THRESH_PUFF` — but read the warning below first, because on sip-and-puff this often makes things **worse**, not better.

Raising the thresholds is a trap for sip-and-puff users. In theory it should help: the thresholds set the *width of the idle band*, so a wider band means pressure re-enters idle sooner on the way back from each element, separating a repeated pair more cleanly. In practice it backfires unless you have a lot of headroom, because the same change also makes every element take **longer to cross the threshold on the way up**, and marginal elements stop registering at all. Breath pressure is modest and slow-changing compared with a switch closure, so most sip-and-puff users have little headroom to spend — the onset penalty outweighs the gap benefit and the error rate climbs. Field result from the project's primary user: steps 1 and 2 fixed the fault; raising the thresholds from 2 to 3 made it noticeably worse.

Only try this if `test_pressure.py` shows your peaks are **several times** your current threshold, and change it back promptly if the error rate does not improve.

Do **not** reach for `ACCEPT_DELAY` for this fault. That setting governs the gap *between characters*; this failure happens *within* a single character, and lowering or raising it will not help.

If you still can't get there: consider `SWITCH_MODE = 3` ("Input modes"). The third-switch Accept gesture commits the character **instantly**, so `ACCEPT_DELAY` stops being a speed limit altogether — you never wait on a timeout. It costs one extra gesture per character, which is a good trade for users who are fast enough that the inter-character pause dominates their typing time. This is also the closest match to Darci USB's end-of-character behaviour.

Display

Display is blank or white - Check that you are running the correct version of `code.py` for your display. The v2 code expects the OLED #326 by default. If you have a TFT FeatherWing, the display initialisation in `code.py` must be updated. - For the OLED: check both ends of both STEMMA QT cables are fully clicked in. - Try changing `device_address=0x3C` to `device_address=0x3D` if the OLED stays blank — some OLEDs use the alternate address.

Speaker

No sound from the STEMMA Speaker (or any speaker option) - Verify wiring: Audio → A0, VIN → 3V, GND → GND. - Confirm the #4046 cable is fully seated in the #3885 speaker's JST PH socket — this is the most common cause of silent speakers. - Open the serial console in Thonny (\$9.3). If you see `WARNING: pwmio module not available`, you are running an unusual minimal CircuitPython build — `pwmio` is present in every standard build for every supported board. Re-flash with the standard `.uf2` from circuitpython.org/downloads. - The audio is a square wave (not a sine), so the tone will sound slightly buzzy — that is expected and not a defect.

Wireless display (ESP-NOW)

Wireless display shows "No signal" - Confirm both boards are powered and running CircuitPython 9.x or later. - Check the serial console on the main board. It should print `ESP-NOW: wireless display active (broadcast)` at startup. - `ESP-NOW: disabled by USE_WIRELESS_DISPLAY = False` → set `USE_WIRELESS_DISPLAY = True` in `config.py` (§10). It ships `False` since most builds have no receiver. - `ESP-NOW: module not available on this board` → the board is not an ESP32-family chip. Wireless display requires ESP-NOW, which is ESP32-only. - `ESP-NOW: init failed (...)` → see entry below. - Range is approximately 30 m indoors. Move the boards closer to test. - Channel mismatch: see "How the ESP-NOW channel is selected" below.

Wireless display resets itself / you see it reboot - This is normal, intended behaviour — not a fault.

The ESP32 WiFi radio on the receiver occasionally wedges: packets stop arriving even though both boards are fine. The receiver detects this and **soft-resets itself after 30 s with no signal** so nobody has to unplug it. The timeline on the receiver is: - **0–10 s** no packets → nothing shown (brief gaps are normal). - **10 s** → the display switches to **"No signal"**. - **30 s** → the receiver **soft-resets** (`microcontroller.reset()`); `boot.py` + `code.py` rerun and, if the sender is up, the mirror is back within a couple of seconds. - **The receiver only auto-resets once it has actually received signal since powering on.** If the sender has never been present this boot — for example the receiver is left plugged into a laptop that is powered **off overnight** — the receiver shows "No signal" and then waits quietly. It does **not** reboot every 30 s all night. When the sender comes back (laptop powers on, main board boots), the receiver is still listening and starts mirroring again on its own with no reset needed. - **If you see it reboot once when the host computer shuts down**, that is expected: the receiver had signal earlier, so it makes one recovery attempt, then the fresh boot has no signal and it goes quiet. - **If the receiver reboot-loops continuously while the sender IS up and running**, that points at a persistent link problem rather than an occasional wedge — check the channel match and range (below), and watch the sender's serial console to confirm it is actually broadcasting. - To change or disable this, edit the receiver firmware: `_AUTO_RESET` in `receiver.py` (Option W1) or `AUTO_RESET_TIMEOUT` in `receiver_magtag.py` (Option W2). Raise the value to reset less eagerly, or set it very high to effectively disable auto-reset.

How the ESP-NOW channel is selected

In CircuitPython 9.x `wifi.radio.channel` is not settable (and on 9.2.9 it isn't even readable). The only reliable way to pin the radio to a specific channel is the **start_ap / stop_ap dance** — start a soft AP on the desired channel, then immediately stop it. The radio stays on that channel. Both boards do this with the same channel:

```
wifi.radio.start_ap(" ", "", channel=ESPNOW_CHANNEL, max_connections=0)
wifi.radio.stop_ap()
espnw.ESPNow()
```

The channel is controlled by `ESPNOW_CHANNEL` in `config.py` (sender) and `ESPNOW_CHANNEL` in `receiver_config.py` (display) — these **must match**. Default is channel 1.

Do **not** use `wifi.radio.connect()` to join a network for channel selection — `connect()` enables WiFi power-save, which silently drops most incoming ESP-NOW frames.

Sender peer setup — the dummy unicast workaround

CircuitPython issue [#9380](#) causes `send()` to a broadcast peer to fail with `ESP_ERR_ESPNOW_NOT_FOUND` (`0x3069`) when **only** the broadcast peer is registered. The fix is to register a dummy unicast peer first, then the broadcast peer:

```
e.peers.append(Peer(mac=b'\x02\x00\x00\x00\x00\x01', channel=ESP_NOW_CHANNEL))
broadcast = Peer(mac=b'\xff\xff\xff\xff\xff\xff', channel=ESP_NOW_CHANNEL)
e.peers.append(broadcast)
e.send(msg, broadcast)
```

The receiver registers **no peers at all** — incoming ESP-NOW frames fire the receive callback regardless of peer registration.

To change the channel (e.g., heavy 2.4 GHz interference on channel 1, common ones to try are 6 and 11):

1. Edit `ESP_NOW_CHANNEL` in `config.py` on the main board. 2. Edit `ESP_NOW_CHANNEL` in `receiver_config.py` on the display to the same value. 3. Save both — each board restarts by itself. Valid 2.4 GHz channels are 1 – 13 (1, 6, 11 are the non-overlapping ones).

Wireless display is frozen / not updating - The receiver updates whenever a packet arrives (10× per second). If the display updates briefly then freezes, the main board may have restarted. Check the main board's serial console for errors.

Main board serial console shows `ESP-NOW: init failed` - The `wifi` module initialisation failed. Try power-cycling the main board. Rarely, the WiFi radio needs a cold boot (unplug power completely rather than pressing Reset).

Repeating `ESP-NOW send error: ESP-NOW error 0x3069` - `0x3069` is `ESP_ERR_ESPNOW_NOT_FOUND` (peer not registered). On CircuitPython 9.x this happens when the sender has registered only a broadcast peer — see [#9380](#). The fix is built into AeroMorse's `code.py`: a dummy unicast peer (`02:00:00:00:00:01`) is registered before the broadcast peer. If you see this error, confirm you have the current `code.py` — the dummy-peer line should appear immediately before the broadcast peer append.

`ESP-NOW: init failed (ESP-NOW error 0x3065)` on CircuitPython 9.0.x - `0x3065` decodes as `ESP_ERR_ESPNOW_NOT_INIT` — the underlying ESP-NOW subsystem wasn't ready when `ESPNow()` was constructed. This is a known bug in CircuitPython 9.0.x that was fixed in 9.1+. Two fixes: - **Easy:** set `USE_WIRELESS_DISPLAY = False` in `code.py` (§10) to skip ESP-NOW entirely. Acceptable if you don't need the wireless display. - **Best:** upgrade CircuitPython on the Feather to **9.2.x or 10.x** from circuitpython.org/downloads. The fix is included in those versions; leave `USE_WIRELESS_DISPLAY = True`.

Wireless display acts as a second keyboard on Windows - The receiver has `morse_map.py` on it. The shared `boot.py` checks for that file to decide its role: present = sender (USB HID enabled), absent = receiver (HID disabled). If you copied `morse_map.py` onto the receiver by mistake, delete it and reboot the receiver — `boot.py` will then leave USB HID off and the host will stop seeing the receiver as a second keyboard.

Option W2 MagTag — e-ink stuck on "[WAITING]" or never updates - ⚠ **First check CircuitPython version.** The 2025 Edition MagTag requires **CircuitPython 10.x or later**. Open `boot_out.txt` on the MagTag's CIRCUITPY drive; if it says 9.x.x the e-ink driver won't work. Re-flash from [circuitpython.org/downloads](https://learn.adafruit.com/adafruit-magtag) with the **MagTag 10.x** UF2. See <https://learn.adafruit.com/adafruit-magtag>. - Confirm `adafruit_magtag/` (the folder, not a single .mpy) is in `/lib`, plus its support libraries (see §9.4.1 Option W2). - Confirm `boot.py` on the MagTag is **the same `boot.py` from the sender** (it auto-detects the receiver role from the absence of `morse_map.py`). If you accidentally copied `morse_map.py` onto the MagTag, `boot.py` will treat it as a sender and enable USB HID — delete the file and reboot. - Note: once the MagTag has mirrored at least once, it **auto-resets after 30 s of lost signal** to recover from a wedged radio — see "Wireless display resets itself" above. A MagTag that has *never* received signal this boot (e.g. sender powered off) will not auto-reset; it waits quietly until the sender returns.

Option W2 MagTag — pattern preview and pressure bar never appear - That's **by design**, not a bug. E-ink refresh is too slow (~1–2 s per redraw) to render the 10 Hz pattern preview or pressure bar meaningfully. The MagTag receiver shows only the group name, last action, and modifiers — the fields that don't change at 10 Hz. For live preview viewing use Option W1 (second #5691 colour TFT).

Option W2 MagTag — last action lags the actual typing by 1–2 seconds - Also by design. The MagTag receiver throttles e-ink refreshes to once every ~2 s so a fast burst of typing doesn't queue a backlog of refreshes. Only the last state at the moment the throttle expires is drawn. If you need live keystroke feedback use Option W1.

CircuitPython versions on the two boards - ESP-NOW sends plain bytes with no awareness of CircuitPython versions. Each board runs its own CircuitPython and its own `/lib` folder. You can have CP9 on the main board and CP10 on the receiver, or any combination — no steps are needed to make them work together. The only requirement is that **each board runs at least CP9** (earlier versions did not include the `espnw` module). There is no need to match versions, update one when you update the other, or do anything differently when the versions differ. - **However:** if you copy the `/lib` folder from the main board to the receiver, both boards must be on the **same major version** (both CP9 or both CP10) — `.mpy` library files compiled for one major version will not load under the other. If the boards are on different major versions, download the matching library bundle for each board separately from circuitpython.org and copy the files individually. The receiver only needs one folder from the bundle: `adafruit_display_text/`.

Software / general

REPL shows `ImportError` - A library file is missing from `lib/`. Re-read §9.2 and copy the missing file from the CircuitPython bundle. The most common one to miss is `adafruit_display_text/` — see the callout in

§9.2.

Nothing happens on the computer at all - Unplug and replug the USB cable. - Confirm the cable is a data cable (a drive should appear when plugged in). - Confirm `boot.py` is on the root of the CIRCUITPY drive. - Confirm `morse_map.py` is named **exactly** `morse_map.py` (not `morse_map_darci.py` or similar) — `code.py` imports it by literal name. See §9.4.

Appendix A — Groups and Group Cycling

AeroMorse has ten groups: g0 (always-available) plus g1–g9. Group 0 is checked in the background on every keystroke (its 8-symbol patterns work in any group). Groups 1–9 cycle with long-press, or you can jump straight to any group with its 8-symbol Group 0 toggle code.

Group	Contents	8-symbol jump code	How to reach by cycling
0	Group-jump patterns (8 symbols)	—	Always active
1	Letters, numbers, punctuation, function keys	<code>.....</code>	Power-on default
2	Mouse movement, clicks, Windows shortcuts	<code>-----</code>	Long-puff cycling
3	Macro text strings	<code>....----</code>	Long-puff cycling
4	Scanning — Enter, Space and F3–F12 on the 12 shortest codes (Switch Control on iOS / Android)	<code>.....-</code> or <code>----....</code>	Long-puff cycling
5	Media — USB HID Consumer Controls (volume / play-pause / mute / track / brightness / eject) on the 12 shortest codes	<code>.....--</code>	Long-puff cycling
6	Placeholder	<code>.....---</code>	Long-puff cycling
7	Switch — sip holds Enter, puff holds Space (games / switch apps; see <code>AEROMORSE_SWITCH_MODE_GUIDE.md</code>). Returns to Keyboard after 20 s idle	<code>...-----</code>	Only with <code>...-----</code> — long-press cycling skips it; leaves by itself
8	Placeholder	<code>..-----</code>	Long-puff cycling
9	Placeholder	<code>.-----</code>	Long-puff cycling

Long sip (hold ≥ `LONG_PRESS` seconds) → cycle backward (...3→2→1→9→8...) **Long puff** (hold ≥ `LONG_PRESS` seconds) → cycle forward (1→2→3→4→...→9→1...)

With 10 groups, cycling all the way around takes a while — the 8-symbol Group 0 jump codes are the fast path to any group from anywhere. The group-toggle codes use a "count of trailing dashes" scheme: 8 dots = g1, then add trailing dashes to reach the higher groups (see `morse_map.py` Group 0 comments for the full list). Group 4 (Scanning) sends F1–F12, making AeroMorse usable as a Switch Control scanning input on iOS and Android, where function keys act as switch actions.

Appendix B — Display Layout

[Keyboard]		← active group name
. - . .		← Morse pattern building up
"n"		← last character or action
Ctrl		← armed modifier / status
		← pressure bar (sensor mode only)

Status line shows: **Ctrl / Shift / Alt / GUI** (sticky modifier armed), **SLOW / FAST / DRAG / RPT** (mouse state).

Appendix C — Customising Macros

Open `morse_map.py`. Find Group 3 near the bottom. Replace the empty strings with your own phrases:

```
g3[3][0b100] = 'Jane Smith'      # -..  same pattern as letter D
g3[4][0b0010] = 'Thank you!'    # ..-  same pattern as letter F
g3[3][0b010] = 'Best regards,'  # .-.  same pattern as letter R
```

Save the file — the Feather reloads automatically.

Appendix D — Breadboard Wiring Walkthrough

Detailed step-by-step instructions for **\$4 Option B1 — Solderless breadboard**. Skip this appendix if you chose Option B2 (TRRS breakout) or Option B3 (#2915 Terminal Block).

Parts — exactly what to buy

Qty	Item	URL	Notes
1	Half-Size Breadboard	https://www.adafruit.com/product/64	Any equivalent generic breadboard also works
1	Breadboard-Friendly 3.5 mm Stereo Jack	https://www.adafruit.com/product/1699	1 jack only — the stereo jack handles both switches
1 pack	Male-to-male jumper wires	https://www.adafruit.com/product/153	You will use 3 of them
1	3.5 mm mono Y-splitter	any store	Needed only if you use 2 switches

No soldering iron. No solder. No stripped wires. All connections are push-in.

Understanding the breadboard

A breadboard is a plastic block full of small holes. Metal clips inside connect certain holes together so you can make circuits by pushing wires into holes instead of soldering.

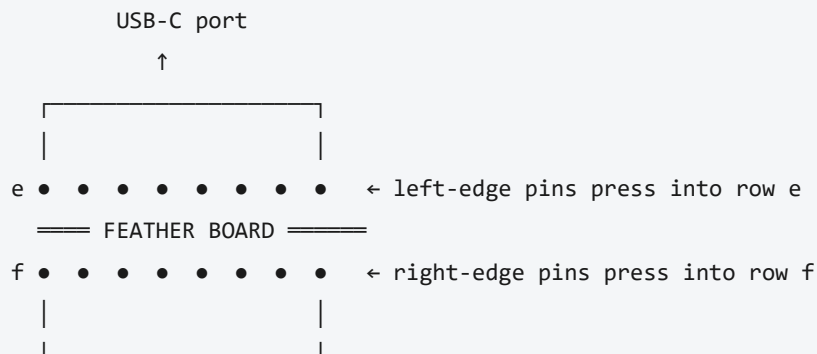
```
← column numbers →
  1  2  3  4  5  6  7  8  9 ...
a [ ][ ][ ][ ][ ][ ][ ][ ][ ]
b [ ][ ][ ][ ][ ][ ][ ][ ][ ] ← holes a-e in the SAME column
c [ ][ ][ ][ ][ ][ ][ ][ ][ ]   are all connected to each other
d [ ][ ][ ][ ][ ][ ][ ][ ][ ]   (5 holes share one clip inside)
e [ ][ ][ ][ ][ ][ ][ ][ ][ ]
  ─────────── ← CENTER GAP (nothing crosses here)
f [ ][ ][ ][ ][ ][ ][ ][ ][ ] ← holes f-j in the SAME column
g [ ][ ][ ][ ][ ][ ][ ][ ][ ]   are connected to each other
h [ ][ ][ ][ ][ ][ ][ ][ ][ ]   (separate from the a-e group)
i [ ][ ][ ][ ][ ][ ][ ][ ][ ]
j [ ][ ][ ][ ][ ][ ][ ][ ][ ]
```

Key rule: Holes sharing the same column number AND same side (a–e, or f–j) are connected. Push a wire into column 5 row c and a pin into column 5 row e — they are now electrically connected without any other wire needed.

The center gap is a physical break — nothing crosses it automatically. That is where the Feather board will sit, bridging the two sides.

Step 1 — Place the Feather in the breadboard

The Feather is a long narrow board with a row of pins along each long edge. Press it firmly into the breadboard so it sits across the center gap like a bridge. One row of pins will be in the **e** row, the other in the **f** row.



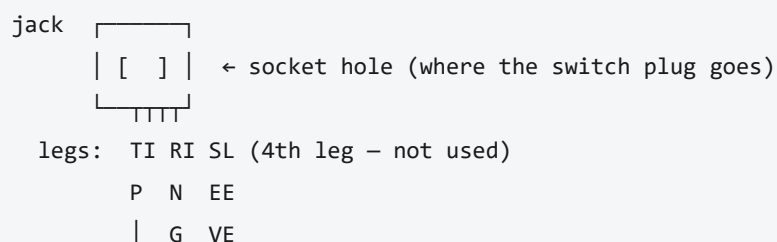
The board should sit level and firm. Every pin in row e now has 4 open holes (a, b, c, d in the same column) you can use to connect to it. Same for row f.

Finding pin D5, D6, and GND on the Feather: Pin names are printed on the *underside* of the Feather board. Before pressing it into the breadboard, flip it over and look. You will see small text next to each pad. On the #5691, D5 and D6 are on the right-side column of pins; GND appears on both sides. Make a small note of which column numbers they land in once the board is inserted.

Tip: Photograph the underside before inserting so you can refer back to the pin names while looking at the top.

Step 2 — Place the 3.5 mm stereo jack (#1699)

The jack has 4 short metal legs underneath it. Press it into an empty area of the breadboard, several columns away from the Feather so there is room to work. Each leg goes into a different row.



The three legs you will use are labelled on the jack's product page: - **TIP** — dot switch (switch 1) - **RING** — dash switch (switch 2) - **SLEEVE** — ground (common return for both switches)

Because each leg is in its own row, you have 4 open holes in that row to connect to.

Step 3 — Connect with jumper wires

The jumper wires from Adafruit #153 have a male pin on each end that pushes into a breadboard hole. You are going to connect three pairs of holes:

From (jack side)	To (Feather side)
Any open hole in the TIP leg's row	Any open hole in D5's column, row a–d
Any open hole in the RING leg's row	Any open hole in D6's column, row a–d
Any open hole in the SLEEVE leg's row	Any open hole in GND's column, row a–d

Push one end of a jumper wire into the jack's TIP row, and the other end into D5's column. The wire can be any length — just pick one short enough to lie flat without flopping around. Repeat for RING→D6 and SLEEVE→GND.

Example layout (column numbers illustrative):

```
col:  5   6   7   ... 18 19 20
a   [ ] [ ] [ ]       [ ] [ ] [ ]
b   [ ] [ ] [ ]       [ ] [ ] [ ]
c   [J] [J] [J]       [ ] [ ] [ ] ← jack legs in cols 5, 6, 7 at row c
d   [—————] [ ] ← jumper wires run across
e   [D5][D6][GN]     [ ] [ ] [ ] ← Feather pins in same cols at row e
      ===== GAP =====
f   [D5][D6][GN]     [ ] [ ] [ ] ← (Feather right-edge, mirror of above)
```

The exact column numbers do not matter — what matters is: - The jack leg and the Feather pin you want to connect are at **opposite ends of the same jumper wire** - No two wires accidentally share a hole

Step 4 — Connect your AT switches

You need **one Y-splitter** (3.5 mm mono-to-stereo) to plug two standard AT switches into the single stereo jack:

- Switch 1 (dot) → plugs into the **Tip** side of the splitter
- Switch 2 (dash) → plugs into the **Ring** side of the splitter

If you only have one switch, plug it directly into the jack — it will be your dot (.) switch.

Appendix E — Configuration Reference

Per-setting deep explanations for every entry in `config.py`. §10's Key Settings table is the quick-reference; this appendix is the "what does this actually do and how should I tune it" companion.

Settings are grouped here in the same order as in `config.py` itself.

Input — sensor / switches / thresholds

`USE_SENSOR` (default `True`). `True` selects Option A in the build guide (LPS33HW pressure sensor

4414) — sip is a dot, puff is a dash. `False` selects Option B (two

AT switches on D5 / D6, or whatever you set `DOT_PIN` / `DASH_PIN` to). The Feather still presents the same USB HID Keyboard / Mouse / ConsumerControl interfaces either way; only the input transducer changes.

`THRESH_SIP` (default `2`), `THRESH_PUFF` (default `2`). hPa delta from the calibrated baseline at which a sip or puff is detected. A sip pulls pressure DOWN, a puff pushes it UP; the symbol fires once the delta exceeds the threshold. - Getting false triggers (sips firing when you're not sipping)? Raise both to `5` or `8`. - Light sips going unrecognised? Drop both to `2` or even `1`. - The two thresholds tune independently — many users find their puffs are naturally stronger than their sips and need a higher `THRESH_PUFF` to balance. - Works in concert with `DEBOUNCE_SAMPLES` — see below.

Use `test_pressure.py` to find your personal thresholds. Copy `test_pressure.py` to the CIRCUITPY drive, open the Thonny Shell, press **Ctrl-C** to interrupt `code.py` and reach the `>>>` prompt (*do not reset — that re-runs code.py*), then type `import test_pressure`. The script calibrates a baseline for 1 second, then prints a live pressure-delta bar chart for 30 seconds while you sip and puff normally. At the end it suggests `THRESH_SIP` and `THRESH_PUFF` values based on 60 % of your observed peaks — copy those into `config.py`. To run it again in the same session:
`exec(open('test_pressure.py').read())` (CircuitPython has no `importlib.reload`).

`DEBOUNCE_SAMPLES` (default `3`). *Sensor mode only.* Number of consecutive sensor readings that must agree on a new state before the firmware accepts a state change. At 75 Hz each sample is ~13 ms, so `DEBOUNCE_SAMPLES = 3` means a change must hold for ~40 ms before it takes effect. Filters out brief pressure wobble that would otherwise turn a single dot into dot-dot. - The big win: with debounce, you can use **low** `THRESH_*` values (2 or even 1) and still get rock-solid element detection. That gives you fast response (no need to press hard) AND wobble immunity. - Set to `1` to disable. Switch mode reads digital pins which are already clean, so this value is ignored when `USE_SENSOR = False`.

POINTS_TO_AVERAGE (default `8`). *Sensor mode only*. Reserved. A rolling average of this depth is maintained but the dot/dash threshold comparison uses the **raw** reading, so changing this value currently has **no effect** on responsiveness or on false-trigger rate. Smoothing is handled upstream by the sensor's own hardware low-pass filter (`SENSOR_FILTER_*` below) and downstream by `DEBOUNCE_SAMPLES`. Don't spend tuning time here.

SENSOR_FILTER_ENABLED (default `True`). *Sensor mode only*. Enables the LPS33HW's built-in hardware low-pass filter. Leave it on unless you are chasing the absolute lowest latency and can tolerate a noisier signal (which usually means raising `THRESH_SIP` / `THRESH_PUFF` to compensate).

SENSOR_FILTER_HEAVY (default `True`). *Sensor mode only*. Selects the filter's cutoff. `True` = ODR/20 (3.75 Hz at the 75 Hz sample rate) — quietest, but it costs roughly **40–60 ms of group delay on every press and every release**. `False` = ODR/9 (8.3 Hz), about half the lag for a modest increase in noise.

This is one of the largest single levers on typing speed and it is easy to miss, because the delay is symmetric: it postpones both the start and the end of every element, so the device feels uniformly "behind" rather than obviously broken. Fast Morse users coming from a switch-based device (where switch closure is read with essentially zero latency) usually want `False` here. See **"Typing feels slower than my previous Morse device"** in §12.

BASELINE_DRIFT_S (default `30`). *Sensor mode only*. Time constant in seconds for auto-zero drift correction. The LPS33HW sensor measures **absolute** atmospheric pressure, not pressure- relative-to-anything, so the boot-time baseline goes stale as real ambient pressure changes — from weather systems, HVAC cycling, doors opening, temperature changes, anything. Without correction, after 30–60 minutes the device often starts firing phantom sips or puffs as the stale baseline drifts further and further from current ambient. - While the state machine is **IDLE** (no active sip/puff), baseline drifts slowly toward the current raw reading at this time constant. At 75 Hz sampling, `BASELINE_DRIFT_S = 30` gives a per-sample correction factor of ~ 0.00044 , so a step change in ambient is $\sim 95\%$ corrected in ~ 90 s. - While in **DIT** or **DAH** the baseline is **frozen** — a held sip or puff cannot be absorbed into the baseline, so a sustained element still detects normally. - Set to `0` to disable auto-zero entirely (return to the original fixed-baseline behaviour). Useful if your environment has perfectly stable atmospheric pressure (rare) or for debugging. - Tune lower (`15`) for environments with fast pressure changes (close to an HVAC vent); higher (`60` or `120`) if you find the baseline drifting away during slow deliberate sips.

DOT_PIN / **DASH_PIN** (defaults `board.D5` , `board.D6`). GPIO pins for the AT-switch jacks. Only used when `USE_SENSOR = False`. Wired via 3.5 mm TRRS jack #1699 (B1/B2) or terminal block #2915 (B3) — see Build Guide §4 Option B and §8C.

Input mode — 1 / 2 / 3 switch

See §10 "Input modes — what `SWITCH_MODE` does" for the mode behaviour table.

SWITCH_MODE (default `2`). `1` = single-switch timed (one input; short = dot, long = dash, pause = end-of-letter). `2` = paddle / two-switch (dot input = dot, dash input = dash). `3` = paddle + explicit Accept (long-press of **THIRD_SWITCH_GESTURE** commits immediately).

ONE_SWITCH_INPUT (default `"dot"`). *Mode 1 only*. Which physical input is the sole switch. `"dot"` uses the sip (sensor mode) or D5 jack (switch mode); `"dash"` uses puff / D6.

ONE_SWITCH_DOT_MS (default `200`). *Mode 1 only*. ms boundary between dot and dash. Press $\leq$ this is a dot, longer is a dash. Group cycle still kicks in at **LONG_PRESS** regardless.

THIRD_SWITCH_GESTURE (default `"long_dash"`). *Mode 3 only*. Which long-gesture serves as the explicit Accept. `"long_dash"` makes a long puff (or long-D6) the Accept; `"long_dot"` makes a long sip (or long-D5) the Accept. The OTHER long-gesture cycles groups forward.

Strong sip / strong puff

A distinct gesture from regular dot/dash and from long-press cycle/Accept. Useful for jumping groups or firing a dedicated action without leaving the current group.

Detection in sensor mode: while a sip/puff is held, the firmware tracks peak pressure delta. As soon as it crosses **THRESH_SIP_STRONG** or **THRESH_PUFF_STRONG**, the configured **STRONG_*_ACTION** fires immediately, the pending Morse pattern is cleared, and no dot/dash is emitted for that press.

Detection in switch mode: switches have no intensity, so the equivalent gesture is a **long-press** of the corresponding switch — DIT-side input fires **STRONG_SIP_ACTION**, DAH-side input fires **STRONG_PUFF_ACTION**. The long-press is detected at the existing **LONG_PRESS** duration. Setting either action **overrides** the **LONG_PRESS_CYCLES_GROUP** cycle-on-long-press behaviour for that switch. **THRESH_*_STRONG** values are ignored in switch mode.

STRONG_SIP_ACTION / **STRONG_PUFF_ACTION** (shipped `"group 2"` / `"group 1"`). Command strings fired when the gesture is detected. Same syntax as g0 toggle codes — most commonly `"group N"` to jump to a specific group. The shipped values jump a strong sip to Group 2 (Mouse) and a strong puff to Group 1 (Keyboard). Set either to `" "` to disable that gesture. Honoured only in **SWITCH_MODE** = `2`.

THRESH_SIP_STRONG / **THRESH_PUFF_STRONG** (defaults `15`). *Sensor mode only*. hPa thresholds. Set noticeably higher than the regular **THRESH_SIP** / **THRESH_PUFF** ($3\times$ is a sensible start) so a normal sip can't accidentally trigger a strong gesture.

Timing

ACCEPT_DELAY (default `0.3`). Idle pause (seconds) after the last element before the pattern commits. - Patterns committing before you finish? Raise to `0.5` or `0.7`. - Patterns feeling sluggish? Lower toward `0.2` or `0.15`. - Sip-and-puff users with a comfortable breath rhythm typically land somewhere in **0.3–0.7**; fast, experienced users may prefer `0.2`. - In **SWITCH_MODE** = `3` this is a safety-net timeout; the third-switch gesture commits instantly regardless of **ACCEPT_DELAY**.

LONG_PRESS (default `1.0`). Hold time (seconds) for the long-gesture (cycle / Accept / strong). Raise (e.g. `1.5`) if accidentally triggering on slow elements; lower (e.g. `0.7`) if a deliberate long-press feels too slow.

Code repeat (Darci-style hold-to-repeat)

CODE_REPEAT (default `False`). When `True`, holding DIT or DAH emits a stream of symbols at the configured repeat interval — one symbol on press, then one every `DOT_REPEAT_MS` / `DASH_REPEAT_MS` while held. Release ends the stream. `ACCEPT_DELAY` of idle still commits the accumulated pattern. This is the Darci USB "code repeat" behaviour. Honoured only in `SWITCH_MODE = 2`.

DOT_REPEAT_MS (default `200`) / **DASH_REPEAT_MS** (default `600`). ms per auto-repeated dot / dash. Dash conventionally $\approx 3 \times$ dot but tune to your rhythm.

CODE_REPEAT_MAX (default `8`). Cap on symbols per held stream. Prevents buffer overflow if a hold goes on too long.

LONG_PRESS_CYCLES_GROUP (default `True`). Independent of `CODE_REPEAT`. `True` = holding DIT/DAH for $\geq$ `LONG_PRESS` cycles groups (DIT = back, DAH = forward). `False` = long-press does nothing; switch groups via g0 Morse patterns instead. **Recommended `False` alongside `CODE_REPEAT = True`** so a sustained symbol stream doesn't accidentally trigger a group cycle.

Audio

AUDIO_PIN (default `board.A0`). PWM output pin for the speaker. All speaker options (S1 STEMMA Speaker, S2 jack-and-piezo, S3 PAM8302 amp) wire to A0 / GND. The output is a 50% duty square wave at the chosen frequency — see Build Guide §6.

BEEP_DOT_FREQ (default `1200`) / **BEEP_DASH_FREQ** (default `800`). Hz — dot (sip) and dash (puff) sidetone frequencies. Different pitches make it easier to hear what symbol just fired. Raise both proportionally if a piezo's resonance peak is higher; the STEMMA speaker is reasonably flat across these frequencies.

CONFIRM_FREQ (default `1050`) / **GROUP_FREQ** (default `550`). Hz — short blips fired when an action commits (`CONFIRM_FREQ`) and when groups change (`GROUP_FREQ`).

BEEP_CONFIRM_S (default `0.06`) / **BEEP_GROUP_S** (default `0.14`). Seconds — duration of the confirm and group-change blips.

Mouse (Group 2 movement speeds)

Effective pixels per step = `raw direction × MOUSE_SPEED_* × MOUSE_SPEED_FACTOR`.

MOUSE_SPEED_NORMAL (default `2`) — default speed when group 2 is entered.

MOUSE_SPEED_SLOW (default `1`) — toggled via the `mslow` command.

MOUSE_SPEED_FAST (default `3`) — toggled via the `mfast` command if you map it.

MOUSE_SPEED_FACTOR (default `2`) — overall scale (matches the AirTalker feel that AeroMorse was originally tuned to).

MOUSE_REPEAT_DELAY (default `0.040`). Seconds between repeat ticks when mouse-repeat is active (40 ms = 25 ticks/sec).

MOUSE_CLICK_MOD_DELAY (default `0.030`). Seconds to hold an armed modifier before and after a mouse click. The keyboard and the mouse are two **separate USB HID interfaces**, so the host is free to process their reports out of order. Without a settle delay the click frequently lands before the modifier registers and the host sees a plain click — the reason **Ctrl+click**, **Shift+click**, and **Alt+click** used to do nothing. If modified clicks are still unreliable on your host, raise this to `0.05`.

MOUSE_CLICK_KEEPS_MODS (default `True`). Controls whether an armed modifier survives a mouse click.

- `True` — the modifier stays armed after the click, so **Ctrl+click multi-select** works the way it does for an able-bodied user: arm Ctrl **once**, click each file you want, then toggle Ctrl back off with the same pattern that armed it. The status line keeps showing `RCtrl1` the whole time so you can see it is still held.
- `False` — one-shot behaviour: the modifier clears after a single click, so every modified click costs two patterns (arm, then click).

Selecting several files in Windows File Explorer. In Group 2, arm **Right Ctrl** (`-.-.`), then send **left click** (`.-`) once per file — the modifier stays armed between clicks. When the selection is complete, enter `-.-.` again to release Ctrl.

Explorer also has a **mouse-only** alternative that needs no modifier at all: turn on **View > Show > Item check boxes** (Windows 11) or **View > Item check boxes** (Windows 10). Each item then gets a checkbox, and plain single clicks on the checkboxes accumulate a multi-selection. This is often far less effort than modified clicking and it survives an accidental un-modified click, which would otherwise collapse the whole selection.

MOUSE_CLICK_HOLD (default `0.060`). Seconds the mouse button is held down for each `mclick`. The firmware does an explicit press → hold → release rather than a single instantaneous click, because a **zero-duration click is ignored by Windows scrollbar tracks** and a number of other controls — they need the button held for a few tens of milliseconds to register. This is the reason a `mdrag` toggle would scroll (it holds the button down) while a plain click on the same scrollbar did nothing. ~60 ms is reliable; raise it if scrollbar or button clicks still don't take, lower it toward `0.03` if clicks feel sluggish.

Scrollbar tip. Clicking the scrollbar **track** (the empty area above or below the thumb) scrolls one page; clicking the little **arrows** at the ends scrolls one line. Clicking the **thumb** itself does *not* scroll — you drag it, which on AeroMorse is `mdrag left` (`-.`) to grab, a few `mmove` steps, then `-.` to release. The mouse wheel patterns (`mwheel up` / `mwheel down`) are usually the least effort for scrolling and don't depend on cursor position over the bar at all.

`MOUSE_CLICK_GAP` (default `0.040`). Seconds between the two clicks of a double-click (`mclick left 2 / mclick right 2`). Only used when the click count is 2. The full double-click therefore takes `HOLD + GAP + HOLD` $\approx$ 160 ms, well within Windows' default 500 ms double-click window. Raise it only if double-clicks are being seen as two single clicks.

Repeat exclusions

`NO_REPEAT_KEYS` (default `( "PAGE_UP", "PAGE_DOWN" )`). A tuple of `Keycode` names (as strings) that must never auto-repeat. After you fire one of these, pressing `repeat` does nothing instead of re-firing the key. Page keys are excluded by default because an accidental repeat scrolls far past where you were and you lose your place. Add any others you don't want to repeat, e.g. `( "PAGE_UP", "PAGE_DOWN", "HOME", "END", "ESCAPE", "TAB" )` .

- (v1.20+) An entry can also be a **key combination**: join the names with `+`, e.g. `"ALT+TAB"` , `"GUI+TAB"` or `"RIGHT_CONTROL+RIGHT_ALT+LEFT_ARROW"` . It blocks exactly that combination (keys in any order). A single-key entry such as `"TAB"` only blocks that key pressed on its own, so `SHIFT+TAB` still repeats unless you list it, and `LEFT_ARROW` alone still repeats.
- The **arrow keys are intentionally not** in this list — Up/Down arrow held with `repeat` is a normal, fine-grained way to scroll, and you stop it with any sip/puff.
- Names are matched against `adafruit_hid.Keycode` . A name that isn't a valid keycode is ignored with a warning at boot rather than crashing, and an older `config.py` without this setting falls back to the page-key default.

Display / wireless

`DISPLAY_ROTATION` (default `0`). Display orientation in degrees. `0` = USB-C port on the LEFT side of the display, `180` = USB-C port on the RIGHT. `90` / `270` are also valid but place the text vertically — useful if you've mounted the Feather sideways.

`USE_WIRELESS_DISPLAY` (default `False`). `True` enables the WiFi radio at boot and broadcasts the display state via ESP-NOW for an Option W1 / W2 receiver to mirror. Default is `False` because most builds are a single board — **set it `True` only when you actually have a receiver paired**. Adds roughly 80–100 mA to the current draw while running. Has no effect on non-ESP32 boards — the `espnw` import fails there and the radio stays off regardless.

`ESPNOW_CHANNEL` (default `1`). The 2.4 GHz WiFi channel (1–13) the sender broadcasts on. **It must match `ESPNOW_CHANNEL` in the display's `receiver_config.py`** or the receiver hears nothing. Only relevant when `USE_WIRELESS_DISPLAY = True` . Change it only if channel 1 is congested in your area. See §12 "How the ESP-NOW channel is selected" for the full mechanism and why the channel is pinned the way it is.

Appendix F — Building with #5477 + 3.5" TFT FeatherWing #3651

A consolidated walkthrough for builders pairing the bare-Feather **ESP32-S3 Feather #5477** with the **3.5" TFT FeatherWing Resistive Touch #3651** — the largest readable display AeroMorse supports (480×320 colour). Same chip family as the recommended #5691, so all behaviour (USB HID, ESP-NOW, etc.) is identical; only the display path differs.

The cost of this build relative to #5691 is **one code edit** (a display-init swap) plus an **optional layout retune** that takes better advantage of the much larger screen. Everything else (sensor, speaker, AT switches, ESP-NOW wireless display) is identical to the recommended build.

F.1 When this build is the right pick

Want	This combo
Big, glance-able display with much more text area than #5691's 240×135	✓ 480×320 is ~4.4× the pixel area
Bright, colour, backlit display	✓ HX8357D TFT with built-in backlight
Same input + ESP-NOW capabilities as #5691	✓ ESP32-S3 chip family is identical
Built-in display (no separate display to buy)	X — display is a separate FeatherWing
Display + Feather as a single PCB (panel-mount friendly)	X — two PCBs stacked together
Cheaper than #5691	X — #5477 + #3651 costs more than #5691 alone

Pick this combo if you want the **biggest display** and don't mind a two-board stack or one code-edit step. Pick #5691 instead for the simplest single-board build.

F.2 Parts list

The hardware-side parts list is the standard sensor / speaker / switch options from §7 with these display-path changes:

Qty	Item	Adafruit #	Notes
1	ESP32-S3 Feather 4MB/2MB PSRAM with Headers	#5477	Order the variant with headers pre-installed, OR buy bare + a header strip (#2886) and solder them — see §F.3
1	3.5" TFT FeatherWing Resistive Touch	#3651	The display itself; resistive touch is not used by AeroMorse
—	<i>(everything else from §7 Sensor / Speaker / Input choices)</i>	—	Sensor (#4414 + STEMMA QT cable), speaker (S1/S2/S3), AT switches if not using sensor

You do **not** need any extra wires, cables, or breakouts beyond the two boards above — the FeatherWing plugs directly onto the Feather's headers.

F.3 Header pins on the #5477 — required

5477 ships as a bare PCB with empty through-hole pads. The

FeatherWing has female sockets on its underside that need male header pins on the Feather to grip. The "build without header pins" path documented in §3 does **not** work with FeatherWings — the wing's holes can't grip flat pads, and gripping the through-holes from below is not mechanically reliable.

Three ways to get headers on:

Path	Cost	Effort
Order #5477 with headers pre-installed (Adafruit lists this as a separate variant)	Same as bare #5477	None
Bring #5477 + #2886 header strip to a makerspace / library maker lab / electronics shop	Usually free or a few dollars	Drop off, pick up
Solder them yourself	Cost of an iron if you don't have one	~30 minutes for a beginner

If you don't already own a soldering iron, the **with-headers** variant or the **makerspace** path are the practical options. See §8 "Before you start" for makerspace pointers.

F.4 Hardware assembly

Once headers are on the #5477:

1. **Mount the #3651 onto the #5477.** Hold the FeatherWing above the Feather with the display facing up. Line up the two rows of holes on the FeatherWing with the two rows of male header pins on the Feather. Press firmly and evenly downward until the wing sits flush. The pins should click through all the way.
2. **Connect the sensor** (sip-and-puff builds). The Feather's STEMMA QT port sits on the side edge of the Feather PCB. The FeatherWing's PCB is **above** the STEMMA QT port, so you can plug a STEMMA QT cable from the side without removing the wing. Run the cable to the LPS33HW #4414 as in §8B.
3. **Connect the speaker.** A0 / 3V / GND are passed through to the FeatherWing's top-side pin headers. The STEMMA Speaker #3885 + #4046 cable plugs onto those header pins exactly the same way it would on a bare Feather. See §8D.
4. **Connect AT switches** (switch-mode builds). D5 and D6 are also passed through. Wire as in §8C.

USB-C, the Reset button, and the user button on the Feather all remain accessible. The FeatherWing has cutouts and pin pass-throughs in the standard FeatherWing layout.

F.5 Software setup

Libraries on /lib

The standard /lib set from §9.2 *plus* the HX8357D driver:

```
/lib/
├─ adafruit_hid/                (always)
├─ adafruit_bus_device/         (always)
├─ adafruit_register/           (always)
├─ adafruit_display_text/       (always)
├─ neopixel.mpy                 (always)
├─ adafruit_lps35hw.mpy          (sip-and-puff builds)
└─ adafruit_hx8357.mpy          (THIS COMBO – for the #3651 driver)
```

Project files

The same `boot.py`, `code.py`, `config.py`, and `morse_map.py` as the standard build. Then make the two edits below to `code.py`.

Edit 1 — Display init swap (required)

The root `code.py` initialises the display with:

```
display = board.DISPLAY
```

That works on #5691 / #5483 / #5300 because those Feathers have a built-in TFT wired up at firmware level, so `board.DISPLAY` is auto-populated. The #5477 has no built-in display, so `board.DISPLAY` is `None` and the firmware crashes at the first label-creation call.

Find that line near the top of the display-setup section (just before `def _make_label`) and replace with:

```

import busio
import fourwire
from adafruit_hx8357 import HX8357

displayio.release_displays()
spi = busio.SPI(board.SCK, MOSI=board.MOSI, MISO=board.MISO)
display_bus = fourwire.FourWire(
    spi, command=board.D10, chip_select=board.D9, reset=None
)
display = HX8357(display_bus, width=480, height=320, rotation=DISPLAY_ROTATION)

```

The `D9 = CS` / `D10 = DC` pin assignment is Adafruit's standard FeatherWing convention — those are the pins the #3651's CS and DC traces wire to on the FeatherWing PCB. The `rotation=DISPLAY_ROTATION` reads from `config.py` so the existing `DISPLAY_ROTATION` setting keeps working.

After this edit your CIRCUITPY drive's `code.py` should boot with **[KEYBOARD]** showing on the #3651 TFT — but the text will appear tiny because the layout is still tuned for the much smaller #5691 panel. That's what the next (optional) edit fixes.

Edit 2 — Resize labels and reposition the pressure bar (optional but strongly recommended)

The default layout positions and scales in `_build_display()` were chosen for the 240×135 px #5691 TFT. On the 480×320 px #3651 the labels are perfectly readable but use only the top-left ~quarter of the panel, leaving most of the screen blank.

To use the full 480×320 area, change the values in `_build_display()` from the **left column** (current #5691 defaults) to the **right column** (#3651 layout). Just edit the literal numbers — no structural changes.

Element	#5691 (current)	#3651 (recommended)	What changes
Group name <code>_make_label(...)</code>	<code>scale=2, y=2</code>	<code>scale=5, y=10</code>	Largest text, top of screen
Morse buffer <code>_make_label(...)</code>	<code>scale=2, y=30</code>	<code>scale=4, y=90</code>	Big, easy to follow as you sip/puff
Last action <code>_make_label(...)</code>	<code>scale=2, y=58</code>	<code>scale=4, y=160</code>	Big "what just fired"
Modifier/status <code>_make_label(...)</code>	<code>scale=2, y=86</code>	<code>scale=3, y=230</code>	Smaller — less important info
Pressure bar y-position (both <code>TileGrid</code> calls)	<code>y=126</code>	<code>y=290</code>	Bottom of screen with 22 px margin
Pressure bar height (<code>Bitmap</code> 2nd arg, both bar bitmaps)	<code>8</code>	<code>20</code>	Thicker for visibility from distance

Concretely, your `_build_display()` becomes something like (only the two `TileGrid` `y=` values and the `Bitmap` height change for the bar; the four `_make_label` calls change their `scale` and `y` arguments):

```

lbl_group = _make_label(root, f"[ {_GROUP_NAMES[1]} ]", _GROUP_COLORS[1], 5, 10)
lbl_buf   = _make_label(root, " ",                                0x00FFFF,      4, 90)
lbl_action = _make_label(root, " ",                                0xFFFF00,      4, 160)
lbl_mods   = _make_label(root, " ",                                0xFF8000,      3, 230)

bar_bg_bmp = displayio.Bitmap(display.width - 8, 20, 1) # was height 8
bar_bg_pal = displayio.Palette(1)
bar_bg_pal[0] = 0x202020
root.append(displayio.TileGrid(bar_bg_bmp, pixel_shader=bar_bg_pal, x=4, y=290))

bar_pal = displayio.Palette(2)
bar_pal.make_transparent(0)
bar_pal[1] = 0x00FF00
bar_bmp = displayio.Bitmap(display.width - 8, 20, 2) # was height 8
root.append(displayio.TileGrid(bar_bmp, pixel_shader=bar_pal, x=4, y=290))

```

The pixel-bar painting code further down in `_update_display()` uses `for y in range(8)` — change those `range(8)` calls to `range(20)` to fill the taller bar.

All numbers above are starting points — feel free to tune. A reasonable rule of thumb on the 480×320: terminalio.FONT is 6×12 px at scale 1, so at scale 5 a character is 30×60 px and you fit $480/30 = 16$ characters per line at full width; at scale 4 you fit 20 characters per line.

F.6 Configuration

Once the display is up, configuration is identical to any other build: edit `config.py` per §10 (and Appendix E for the deep notes). Set `USE_SENSOR`, thresholds, switch mode, and so on the same way you would on a #5691. The display layout edits above don't interact with any of those settings.

F.7 Verifying it works

A healthy first boot on this build should show:

```
Baseline: <pressure> sip<...> puff>...
Calibration complete – ready for input.
Input mode: 2-switch (1-sw input = dot, 3-sw accept = long_dash)
Code repeat: off long-press cycles group: True
Baseline auto-zero: 30s time constant
ESP-NOW: disabled by USE_WIRELESS_DISPLAY = False
```

...and the 3.5" TFT should display the standard four-row layout with the pressure bar at the bottom (after applying Edit 2), at the much larger scale.

If you get a blank display or a `KeyError` / `AttributeError` on `board.DISPLAY`, Edit 1 didn't apply cleanly — open the file again and confirm the import lines and the `display = HX8357(...)` block are above the first `_make_label(...)` call.

If the display lights up but text is tiny in the top-left corner of a blank screen, Edit 1 worked but Edit 2 wasn't applied yet. Apply Edit 2 for the full-size layout.

If text rotates the wrong way, change `DISPLAY_ROTATION` in `config.py` (valid values: `0`, `90`, `180`, `270`).

AeroMorse is open-source. Project files at: <https://github.com/jlubin2001/AeroMorse>